

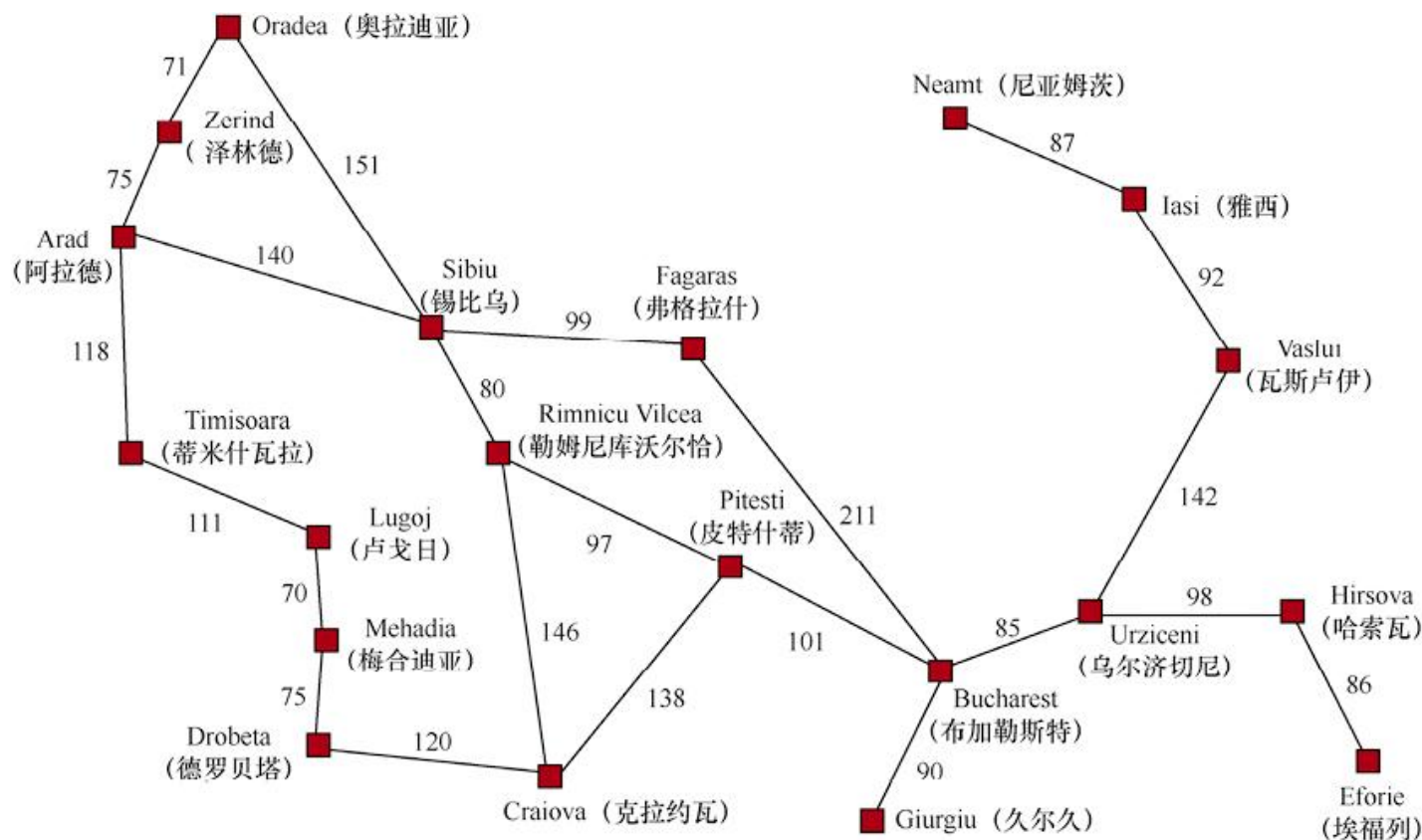
重要知识点回顾

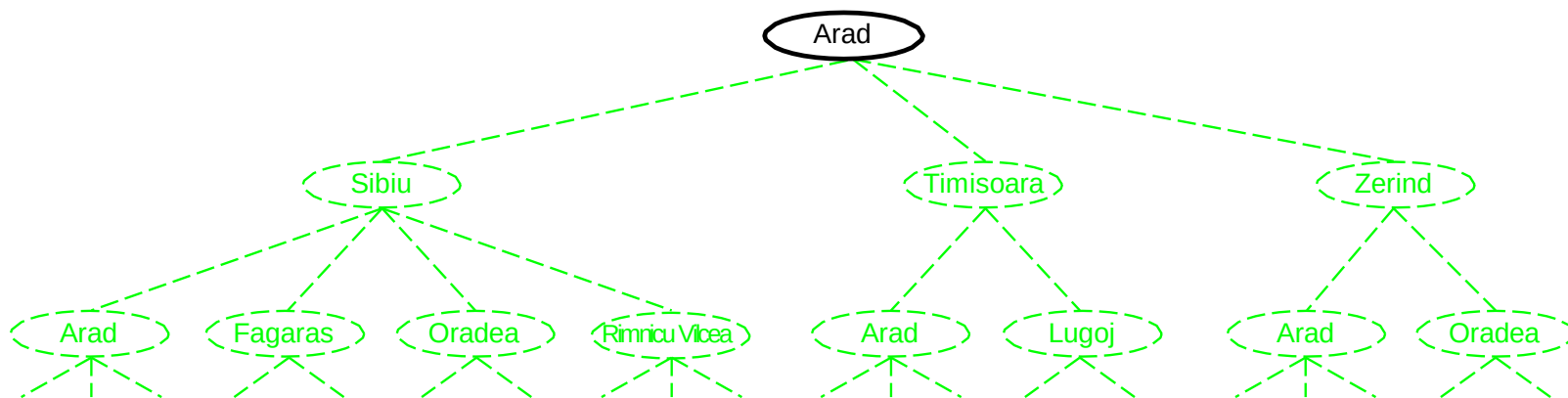
◆ 第三章：通过搜索进行问题求解

- 无信息搜索策略
- 有信息（启发式）搜索策略

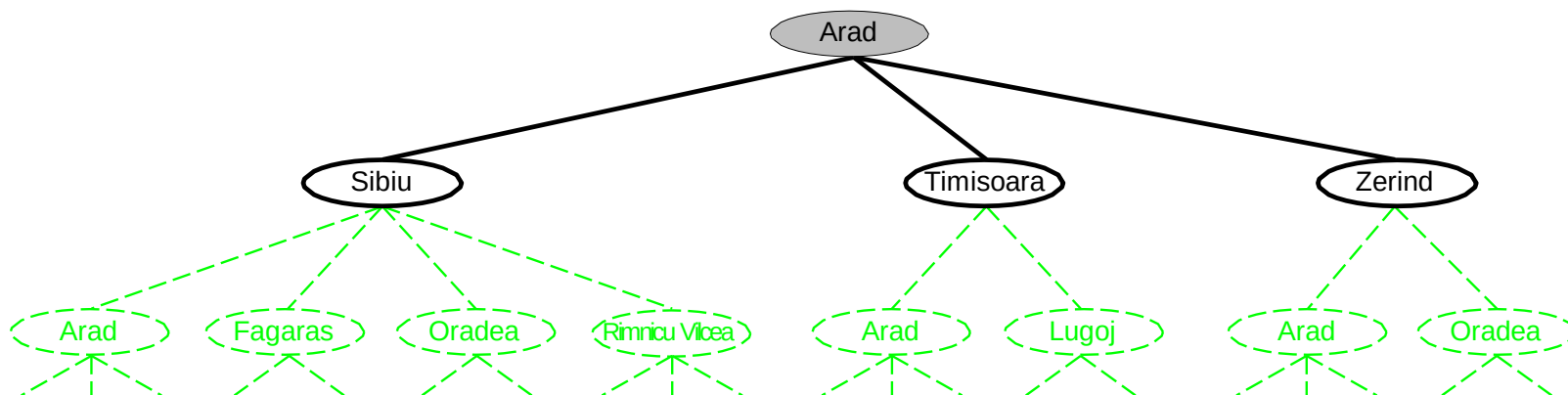
树搜索算法

该算法通过**扩展 (expand)** 节点，从初始状态形成各条路径，并试图找到一条可以达到某个目标状态的路径。搜索树中的每个**节点 (node)** 对应于状态空间中的一个**状态 (state)**，搜索树中的边对应于动作，树的根对应于问题的初始状态。

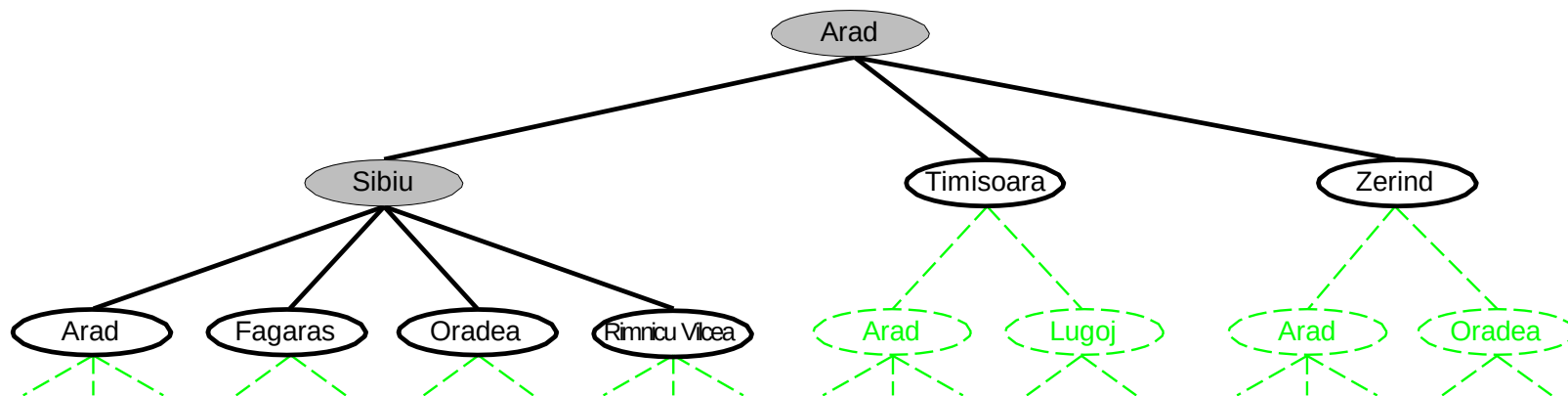




部分搜索树，用于寻找从Arad到Bucharest的路线。

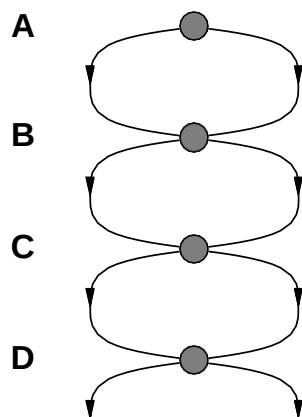


部分搜索树，用于寻找从Arad到Bucharest的路线。

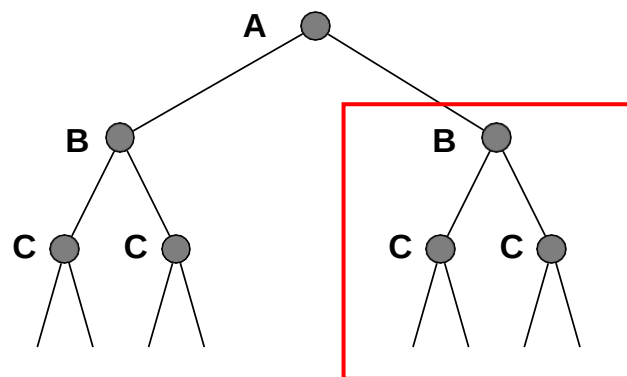


部分搜索树，用于寻找从Arad到Bucharest的路线。

冗余路径



状态空间



冗余路径

搜索树

如果搜索算法检查冗余路径，我们称之为**图搜索 (graph search)**；
否则，称之为**树搜索 (tree-like search)**

最佳优先搜索 (best-first search)

```
function BEST-FIRST-SEARCH(problem, f) returns 一个解节点或 failure  
  node  $\leftarrow$  NODE(STATE=problem.INITIAL)  
  frontier  $\leftarrow$  一个以 f 排序的优先队列, 其中一个元素为 node  
  reached  $\leftarrow$  一个查找表, 其中一个条目的键为 problem.INITIAL, 值为 node  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    if problem.IS-GOAL(node.STATE) then return node  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if s 不在 reached 中 or child.PATH-COST < reached[s].PATH-COST then  
        reached[s]  $\leftarrow$  child  
        将 child 添加到 frontier 中  
  return failure
```

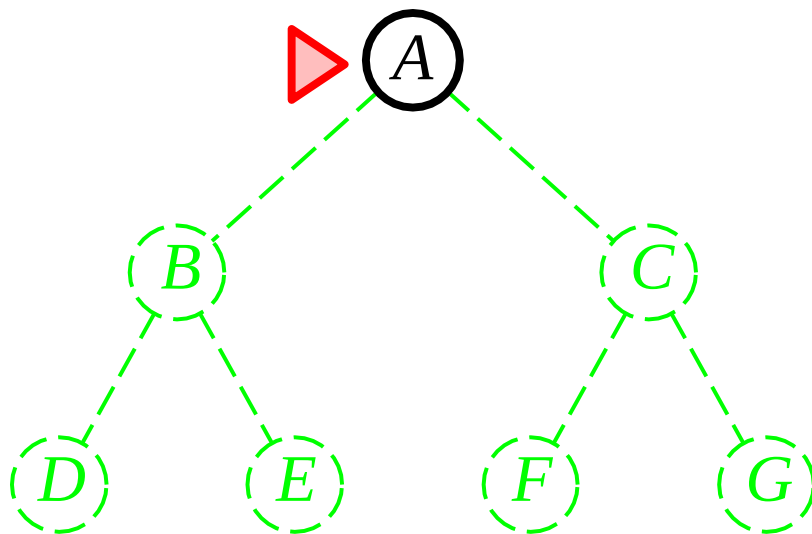
```
function EXPAND(problem, node) yields 节点  
  s  $\leftarrow$  node.STATE  
  for each action in problem.ACTIONS(s) do  
    s'  $\leftarrow$  problem.RESULT(s, action)  
    cost  $\leftarrow$  node.PATH-COST + problem.ACTION-COST(s, action, s')  
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```


无信息搜索算法不提供有关某个状态与目标状态的接近程度的任何线索：

- 广度优先搜索 Breadth-first search
- 一致代价搜索 Uniform-cost search
- 深度优先搜索 Depth-first search
- 深度受限搜索 Depth-limited search
- 迭代加深搜索 Iterative deepening search

广度优先搜索

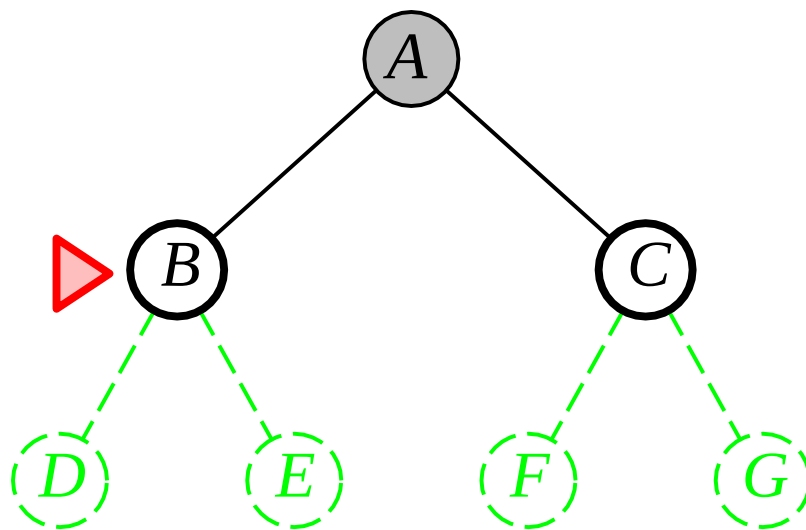
- 优先扩展最浅的未被扩展的节点
- 边界 (frontier) 可以实现为一个FIFO队列, 即, 新节点 (总是比其父节点更深) 进入队列的队尾, 而旧节点, 即比新节点浅的节点, 首先被扩展。



简单二叉树上的广度优先搜索。每个阶段接下来要扩展的节点用三角形标记表示。

广度优先搜索

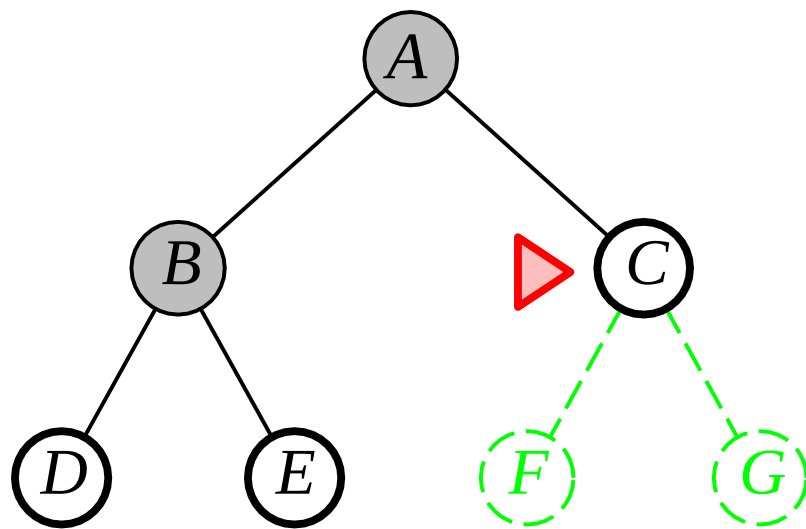
- 优先扩展最浅的未被扩展的节点
- 边界 (frontier) 可以实现为一个FIFO队列, 即, 新节点 (总是比其父节点更深) 进入队列的队尾, 而旧节点, 即比新节点浅的节点, 首先被扩展。



简单二叉树上的广度优先搜索。每个阶段接下来要扩展的节点用三角形标记表示。

广度优先搜索

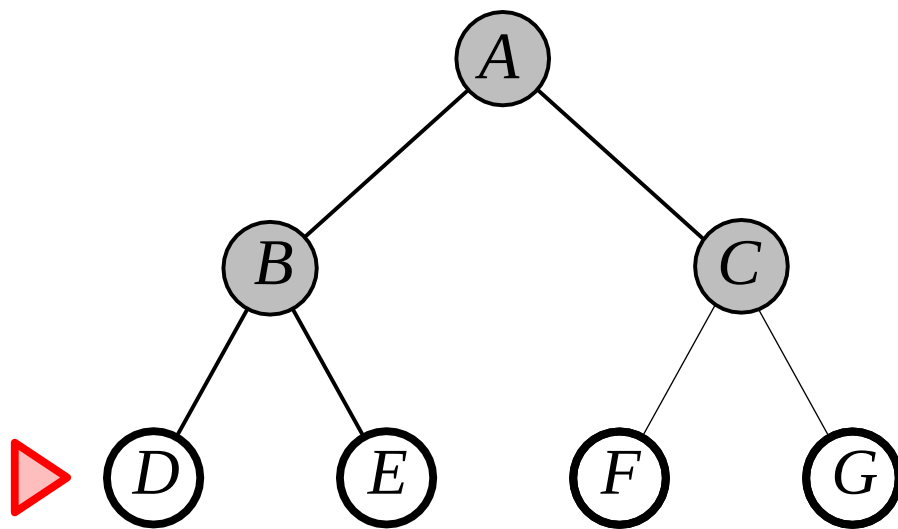
- 优先扩展最浅的未被扩展的节点
- 边界 (frontier) 可以实现为一个FIFO队列, 即, 新节点 (总是比其父节点更深) 进入队列的队尾, 而旧节点, 即比新节点浅的节点, 首先被扩展。



简单二叉树上的广度优先搜索。每个阶段接下来要扩展的节点用三角形标记表示。

广度优先搜索

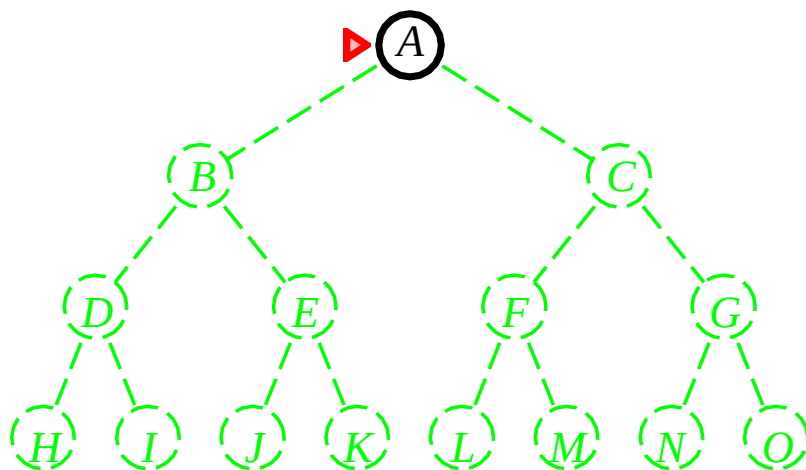
- 优先扩展最浅的未被扩展的节点
- 边界 (frontier) 可以实现为一个FIFO队列, 即, 新节点 (总是比其父节点更深) 进入队列的队尾, 而旧节点, 即比新节点浅的节点, 首先被扩展。



简单二叉树上的广度优先搜索。每个阶段接下来要扩展的节点用三角形标记表示。

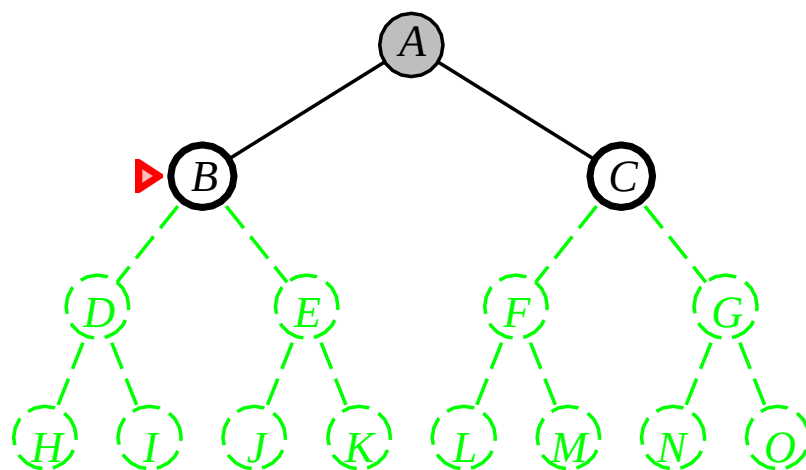
深度优先搜索

- 优先扩展最深的未被扩展的节点
- 边界 (frontier) 可以实现为一个LIFO队列，即后进先出 (Last In, First Out) 。



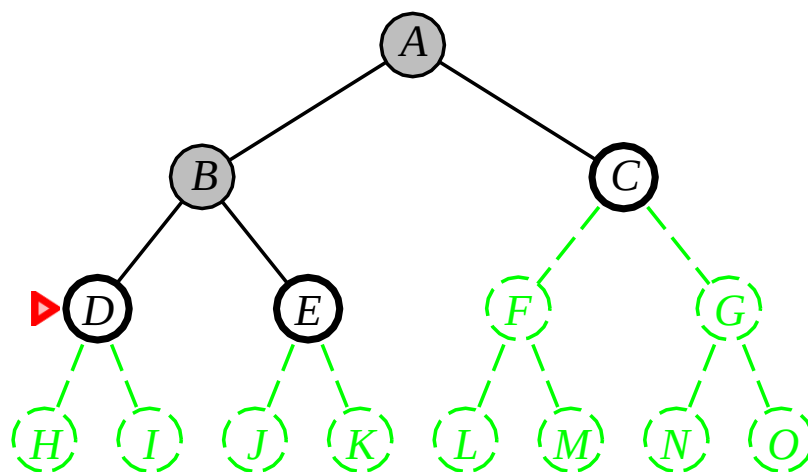
二叉树的深度优先搜索过程中，从开始状态A到目标M

深度优先搜索



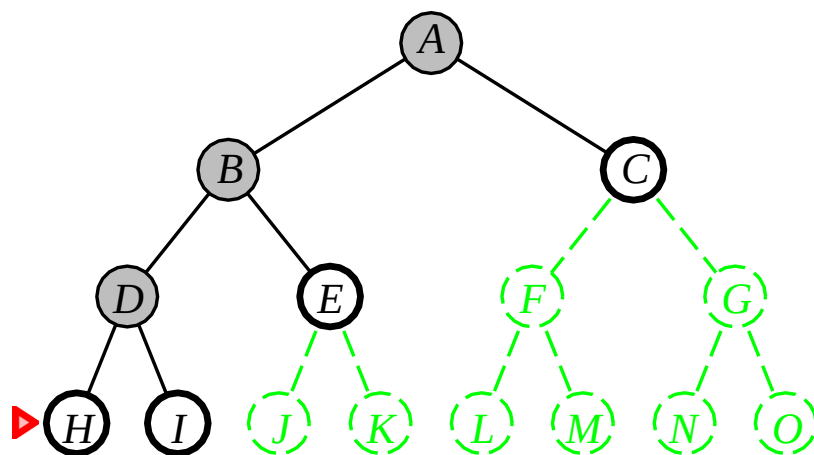
二叉树的深度优先搜索过程中，从开始状态A到目标M

深度优先搜索



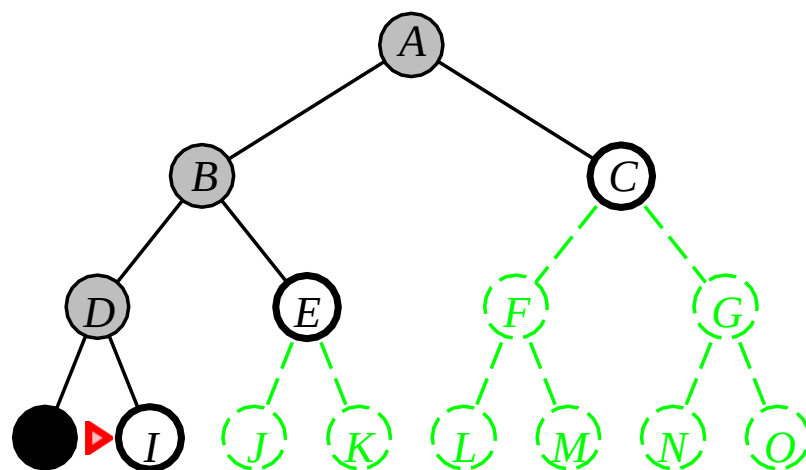
二叉树的深度优先搜索过程中，从开始状态A到目标M

深度优先搜索



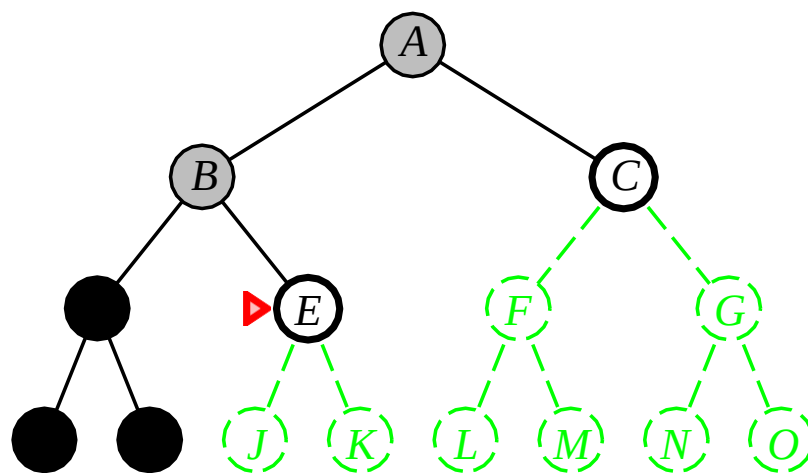
二叉树的深度优先搜索过程中，从开始状态A到目标M

深度优先搜索



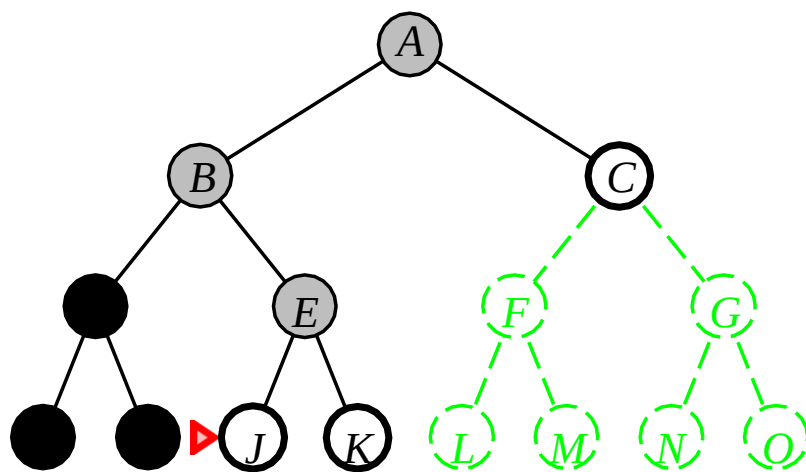
二叉树的深度优先搜索过程中，从开始状态A到目标M

深度优先搜索



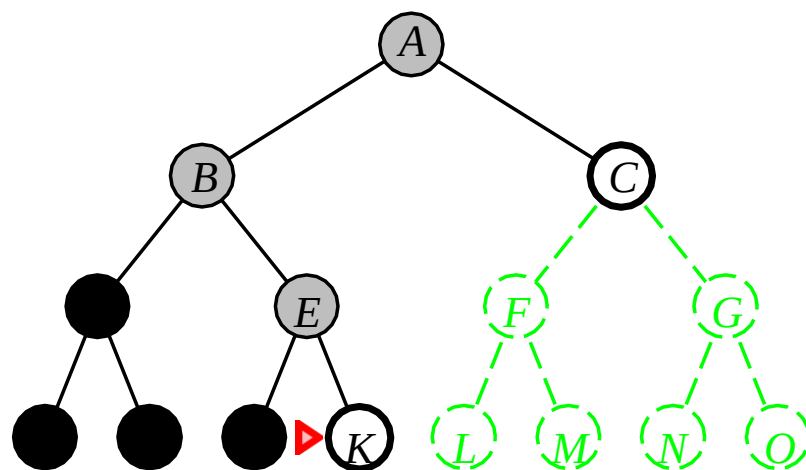
二叉树的深度优先搜索过程中，从开始状态A到目标M

深度优先搜索



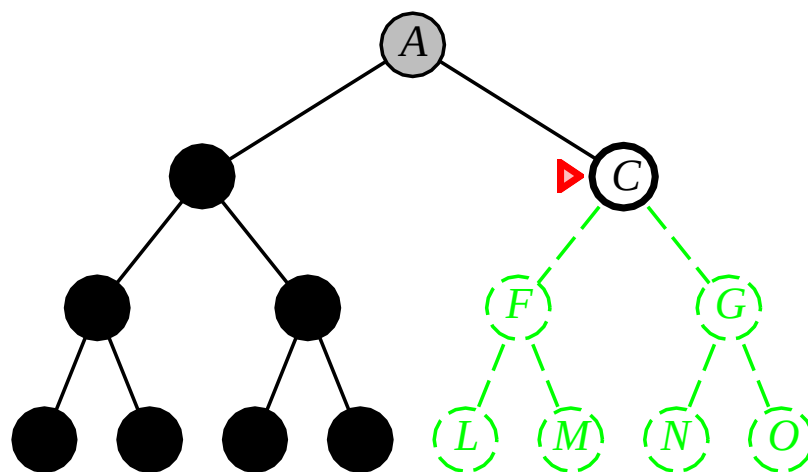
二叉树的深度优先搜索过程中，从开始状态A到目标M

深度优先搜索



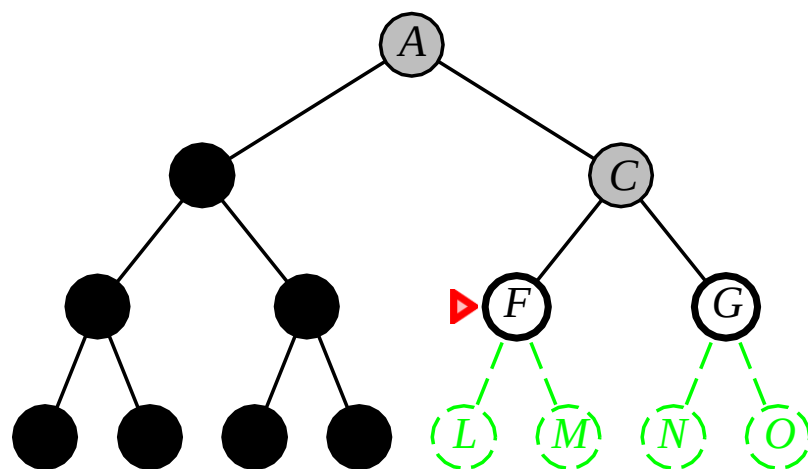
二叉树的深度优先搜索过程中，从开始状态A到目标M

深度优先搜索



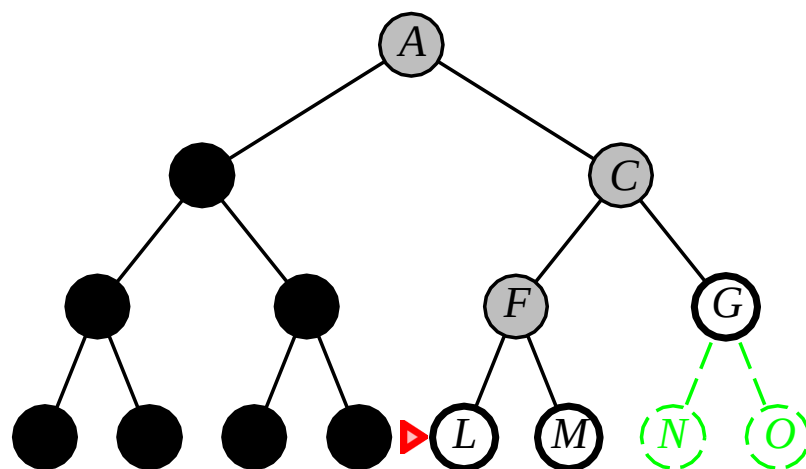
二叉树的深度优先搜索过程中，从开始状态A到目标M

深度优先搜索



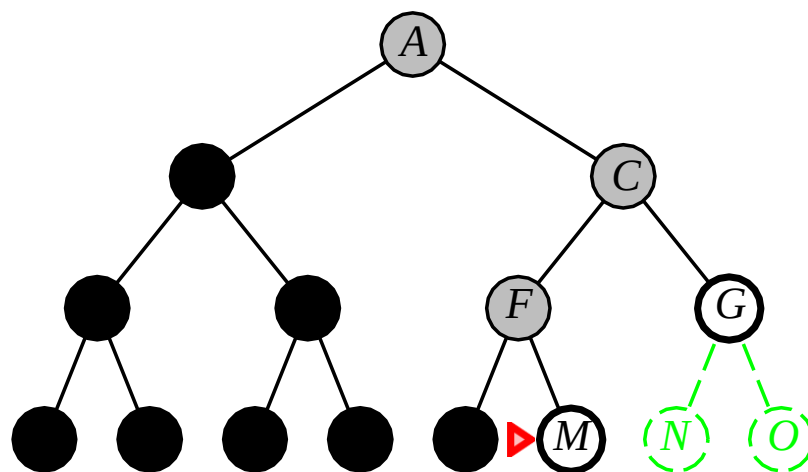
二叉树的深度优先搜索过程中，从开始状态A到目标M

深度优先搜索



二叉树的深度优先搜索过程中，从开始状态A到目标M

深度优先搜索



二叉树的深度优先搜索过程中，从开始状态A到目标M

- **有信息搜索 (informed search) 策略**使用关于目标位置的特定领域线索来比无信息搜索策略更有效地找到解。
- 线索以**启发式函数 (heuristic function)** 的形式出现，记为 $h(n)$:

$h(n)$ = 从节点 n 的状态到目标状态的最小代价路径的代价估计值

例如，在寻径问题中，我们可以通过计算地图上两点之间的直线距离来估计从当前状态到目标的距离。

有信息（启发式）搜索策略

贪心最佳优先搜索（greedy best-first search）

- 最佳优先搜索的一种形式，首先扩展 $h(n)$ 值最小的节点，即看起来最接近目标的节点，因为这样可能可以更快找到解。
- 代价函数 $f(n) = h(n)$

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

h_{SLD} （到布加勒斯特的直线距离）的值

A*搜索

- 最佳优先搜索的一种形式，主要思想：避免扩展路径代价已经很高的路径
- 代价函数 $f(n) = g(n) + h(n)$
 - $g(n)$ = 从初始节点到节点n的路径代价
 - $h(n)$ = 从节点n的状态到目标状态的最小代价路径的代价估计值
 - $f(n)$ = 经过n到一个目标状态的最优路径的代价估计值
- 对于**可容许的启发式 (admissible heuristic)** 函数，A*搜索是代价最优的，即

$$h(n) \leq h^*(n)$$

这里 $h^*(n)$ 经过节点n的状态到目标状态的真实代价. ($h(n) \geq 0$, 对于任意目标状态G, $h(G) = 0$.)

知识点回顾

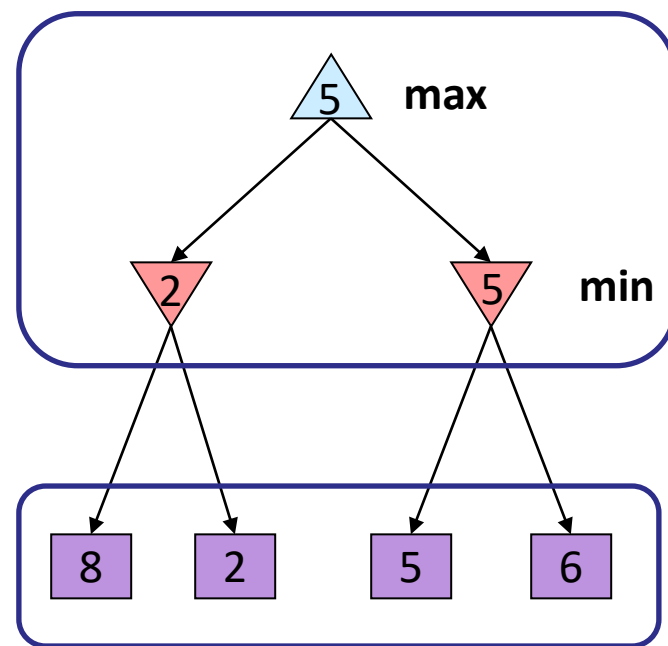
◆ 第五章：对抗搜索和博弈

- 极小化极大搜索
- 评价函数
- α - β 剪枝

极小化极大搜索

- 确定性的，零和游戏：
 - 井字棋, 象棋, 围棋
 - 一个玩家最大化结果
 - 另一个玩家最小化结果
- 极小化极大搜索：
 - 状态空间搜索树
 - 玩家交替进行操作
 - 计算每个节点的**极小化极大值**:
针对对手所能获得的最佳效用值

极小化极大值：
递归计算



终止状态值：
效用值

博弈中的优化决策

极小化极大搜索算法实现

def max-value(state):

 initialize $v = -\infty$

 for each successor of state:

$v = \max(v, \text{min-value}(\text{successor}))$

 return v



def min-value(state):

 initialize $v = +\infty$

 for each successor of state:

$v = \min(v, \text{max-value}(\text{successor}))$

 return v

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

博弈中的优化决策

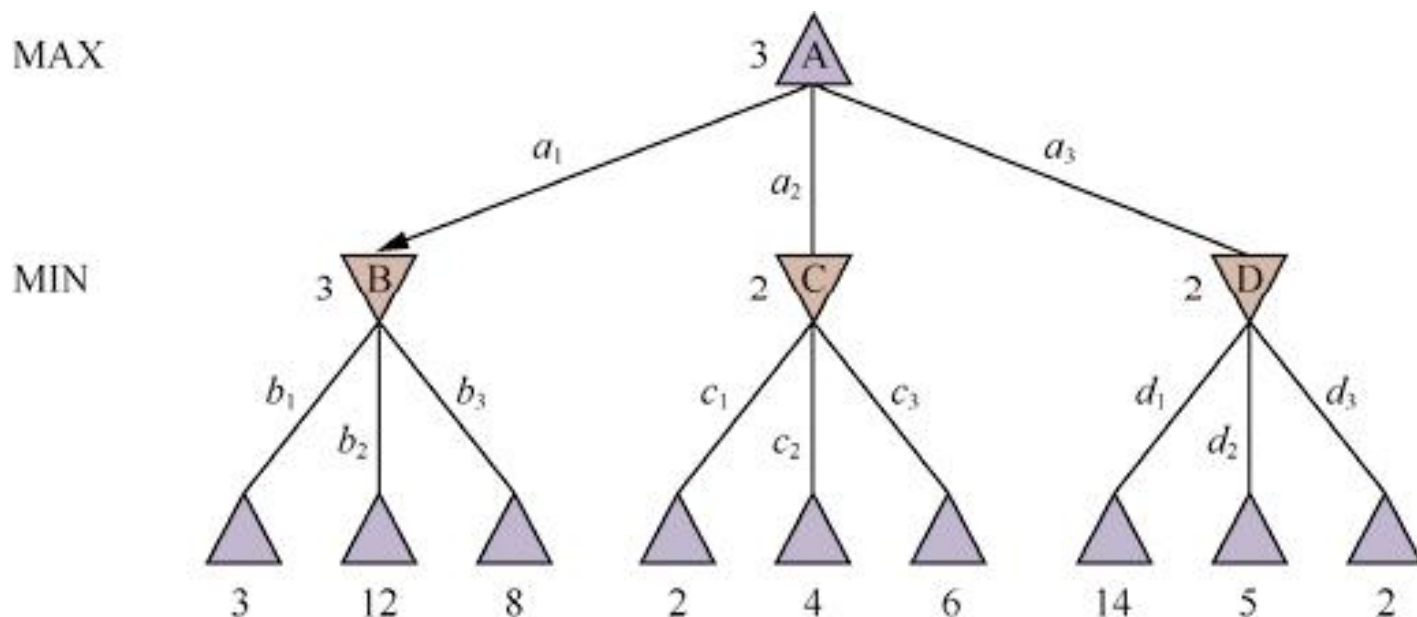
极小化极大搜索算法实现

```
def value(state):  
    if the state is a terminal state: return the state's utility  
    if the next agent is MAX: return max-value(state)  
    if the next agent is MIN: return min-value(state)
```

```
def max-value(state):  
    initialize  $v = -\infty$   
    for each successor of state:  
         $v = \max(v, \text{value}(\text{successor}))$   
    return  $v$ 
```

```
def min-value(state):  
    initialize  $v = +\infty$   
    for each successor of state:  
         $v = \min(v, \text{value}(\text{successor}))$   
    return  $v$ 
```


极小化极大搜索示例



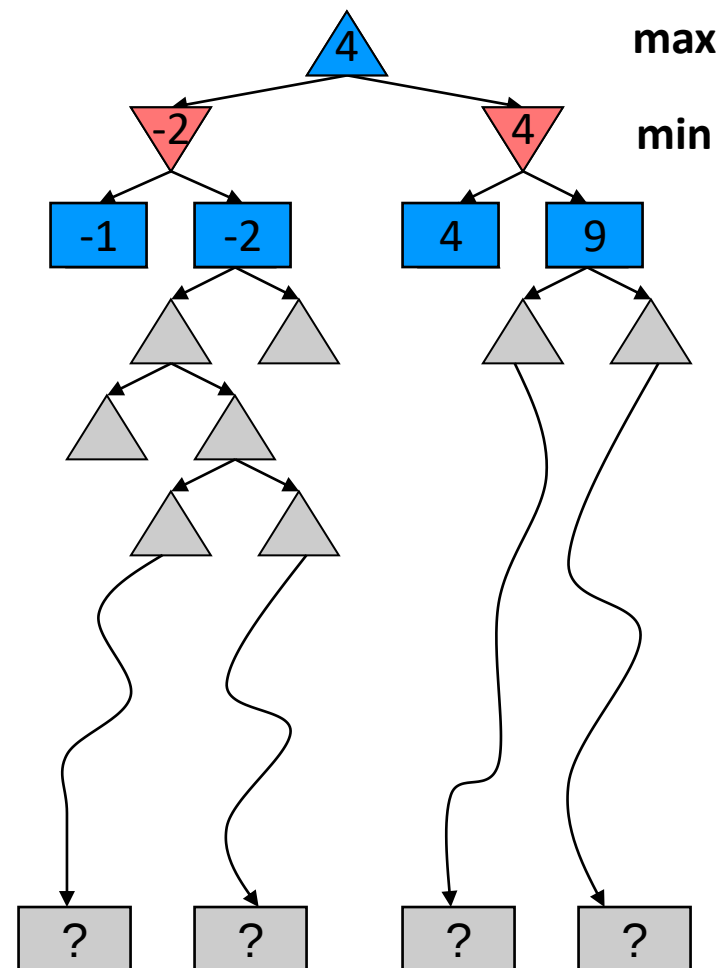
二层博弈树。 Δ 节点为“max节点”，即轮到max移动， ∇ 节点为“min节点”。终止节点显示max的效用值，其他节点标记有它们的极小化极大值。max在根节点的最佳移动是 a_1 ，因为它指向极小化极大值最高的状态，而min的最佳响应是 b_1 ，因为它指向极小化极大值最低的状态。

极小化极大搜索算法实际上是对博弈树进行一个完整的深度优先搜索

博弈中的优化决策

资源受限下的搜索

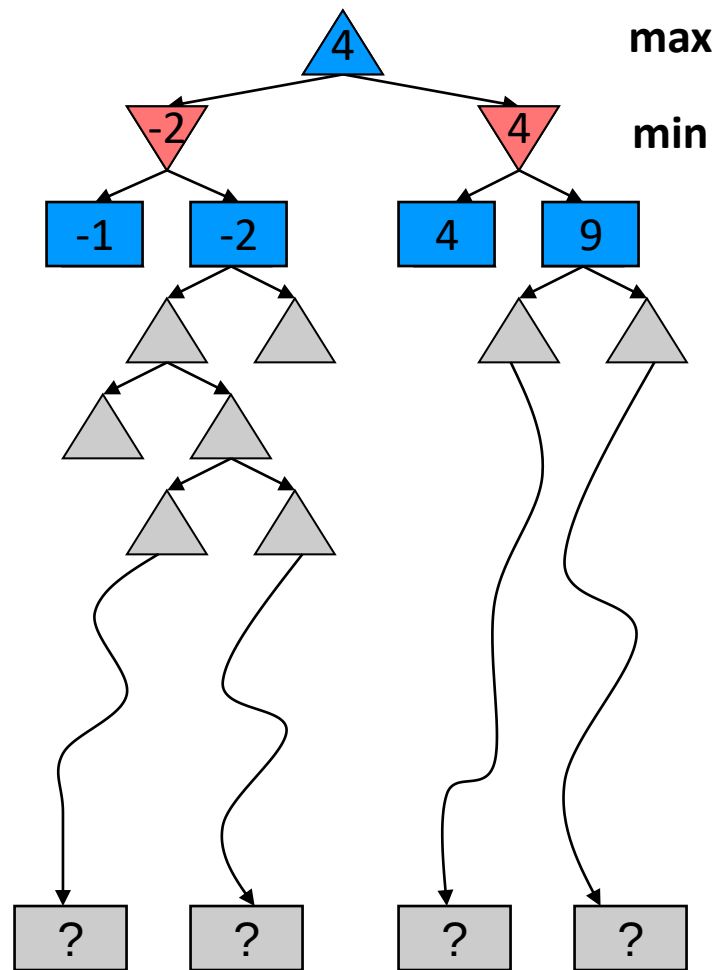
- **问题：** 在真实的博弈当中,无法搜索到叶节点
- **解决方案：** 深度受限搜索
 - 1) 仅搜索到树中的有限深度
 - 2) 对于非终止状态使用**评价函数**替代终止效用函数



博弈中的优化决策

资源受限下的搜索

- **问题：** 在真实的博弈当中,无法搜索到叶节点
- **解决方案：** 深度受限搜索
 - 1) 仅搜索到树中的有限深度
 - 2) 对于非终止状态使用**评价函数**替代终止效用函数
- 无法保证最优性
- 使用迭代加深确定合适的搜索深度



博弈中的优化决策

评价函数

- 在深度受限搜索中评价函数给出非终止状态的期望效用的估计值

$$\text{UTILITY}(loss, p) \leq \text{EVAL}(s, p) \leq \text{UTILITY}(win, p)$$

- 理想的函数：返回位置的实际的极小化极大值
- 在实际中：特征的线性加权，例如：

$$f_1(s) = (\text{白棋皇后数} - \text{黑棋皇后数})$$

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s) = \sum_{i=1}^n w_i f_i(s)$$

博弈中的优化决策

评价函数

- 在深度受限搜索中评价函数给出非终止状态的期望效用的估计值

$$UTILITY(loss, p) \leq EVAL(s, p) \leq UTILITY(win, p)$$

- 理想的函数：返回位置的实际的极小化极大值
- 在实际中：特征的线性加权，例如：

$$f_1(s) = (\text{白棋皇后数} - \text{黑棋皇后数})$$

$$EVAL(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s) = \sum_{i=1}^n w_i f_i(s)$$

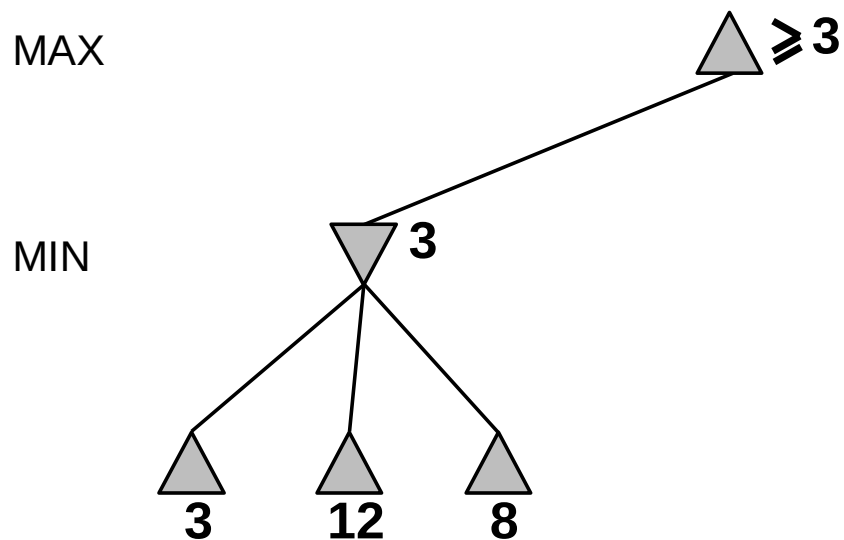
- **一个好的评价函数：**

首先，计算时间不能太长！（加快搜索速度）

其次，评价函数应与实际的获胜机会密切相关。

博弈中的优化决策

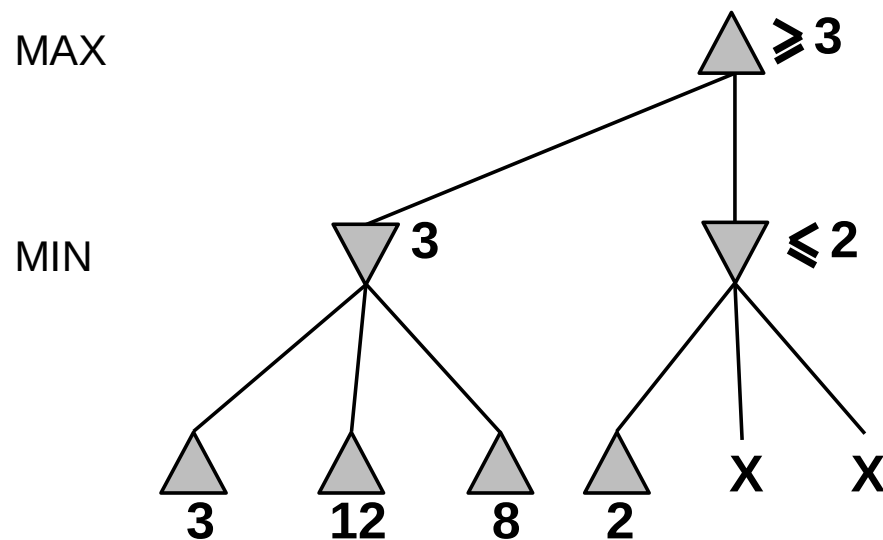
α - β 剪枝 (alpha-beta pruning)



博弈树的最优决策计算过程

博弈中的优化决策

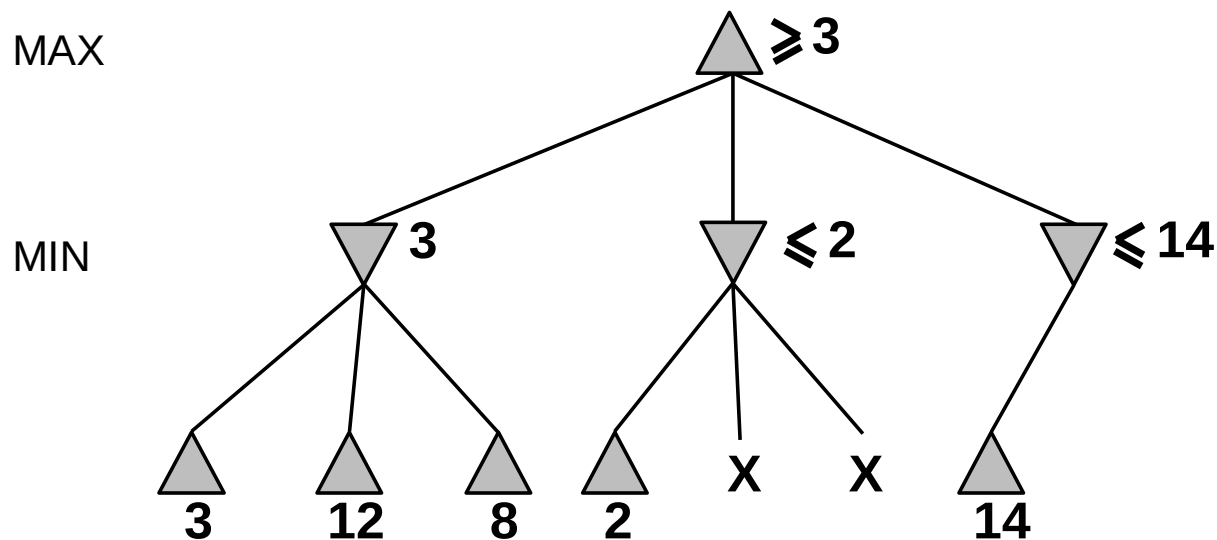
α - β 剪枝 (alpha-beta pruning)



博弈树的最优决策计算过程

博弈中的优化决策

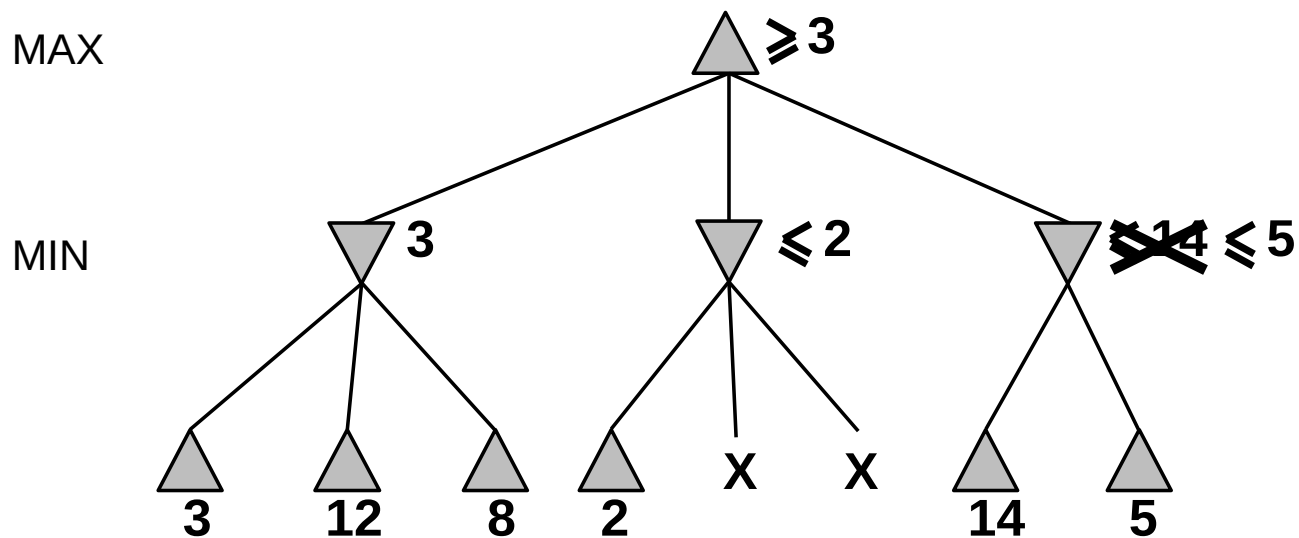
α - β 剪枝 (alpha-beta pruning)



博弈树的最优决策计算过程

博弈中的优化决策

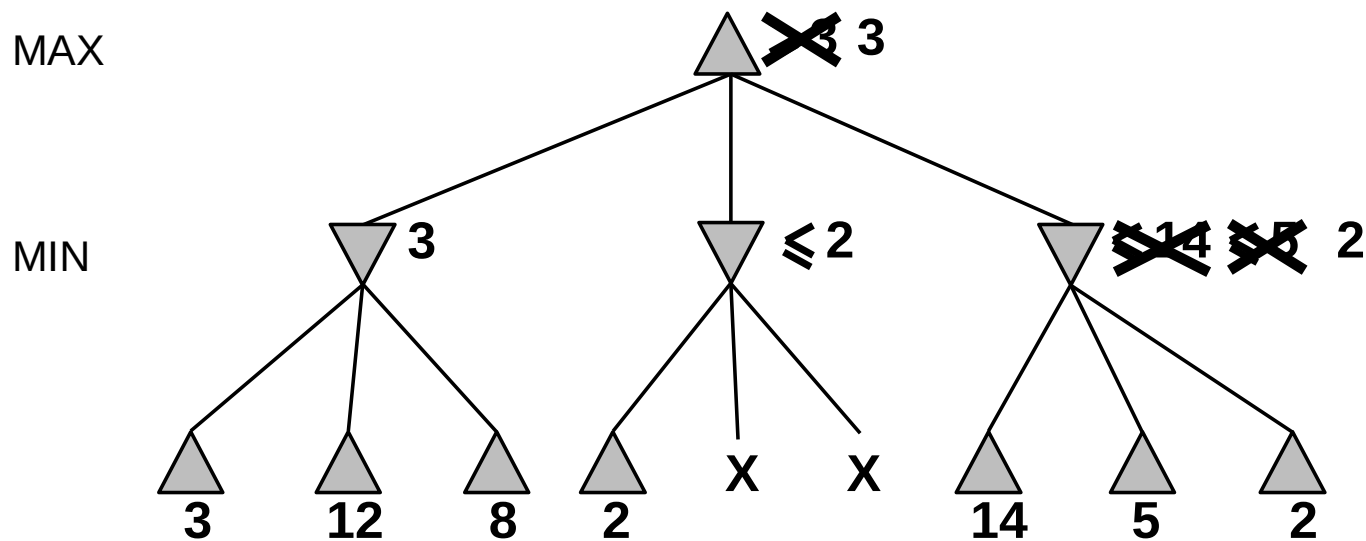
α - β 剪枝 (alpha-beta pruning)



博弈树的最优决策计算过程

博弈中的优化决策

α - β 剪枝 (alpha-beta pruning)



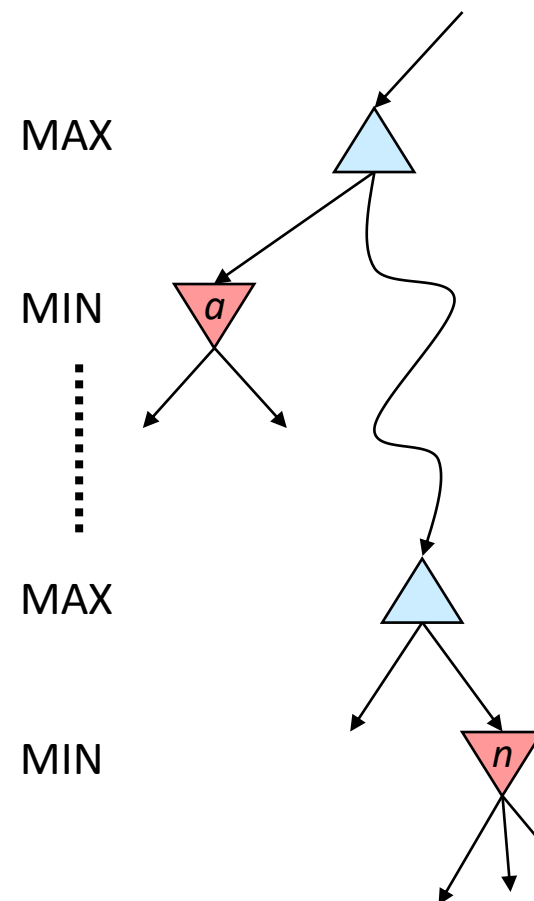
博弈树的最优决策计算过程

博弈中的优化决策

α - β 剪枝 (alpha-beta pruning)

α - β 剪枝的一般情况:

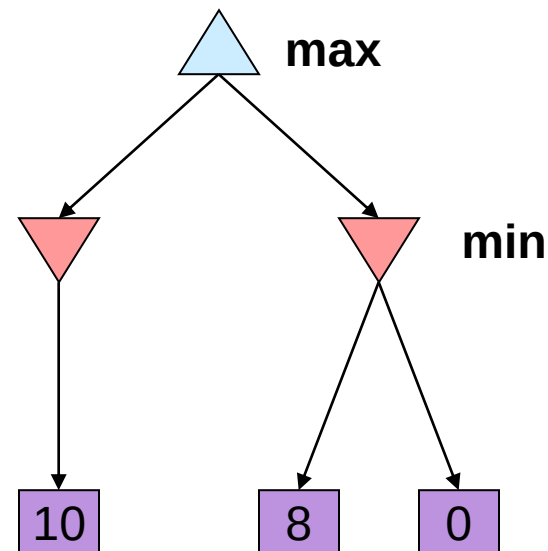
- 计算某一节点 n 的MIN值;
- 开始遍历 n 的后继节点;
- n 节点的值是随着后继节点的遍历而下降的;
- 令 α 为MAX当前能够获得的最大值;
- 如果 n 的值变得比 a 小, 它的值便不会被MAX采用, 所以我们停止考虑 n 的其他后继节点。



博弈中的优化决策

α - β 剪枝 (alpha-beta pruning) 的性质

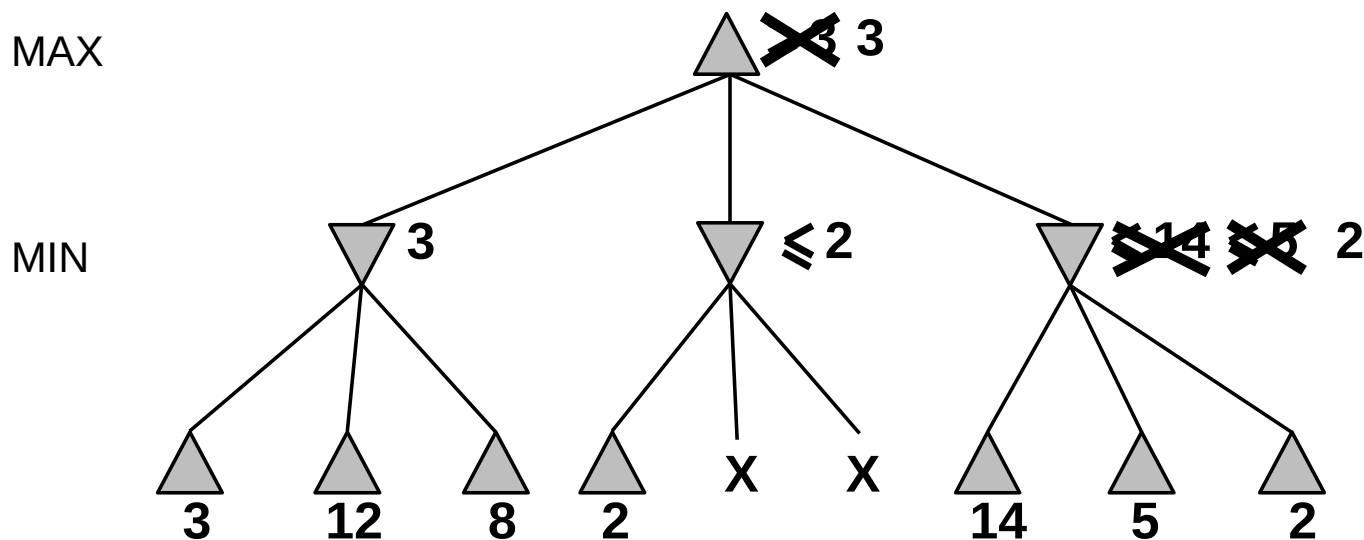
- 剪枝不影响根节点极小化极大值的计算。
- 中间节点的值在剪枝后可能是错误的。
- 好的后继节点遍历顺序会提高剪枝效率。



博弈中的优化决策

α - β 剪枝 (alpha-beta pruning) 的性质

- 剪枝不影响根节点极小化极大值的计算。
- 中间节点的值在剪枝后可能是错误的。
- 好的后继节点遍历顺序会提高剪枝效率。

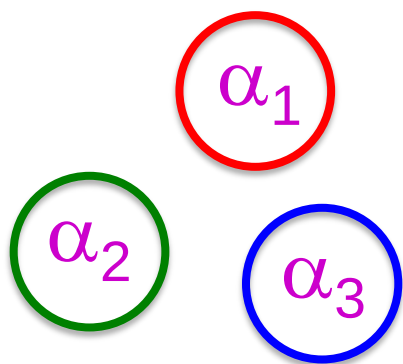


知识点回顾

◆ 第七章：逻辑智能体

- 逻辑推断
- 命题逻辑：语法
- 命题逻辑：语义
- 命题定理证明

- **逻辑 (logic)** : 用于表示信息的形式化语言以便能够对世界进行推理。
- **语法 (syntax)** : 规定了所有的合规语句。
- **语义 (semantics)** : 定义每条语句在每个可能世界中的真值, 即, 用于判定特定模型中语句真值的规则。



语法域



语义域

- 如果语句 α 在模型 m 中为真，我们说 m 满足 α ，有时也可以说 m 是 α 的一个模型。我们使用记号 $M(\alpha)$ 来代表 α 的所有模型的集合。

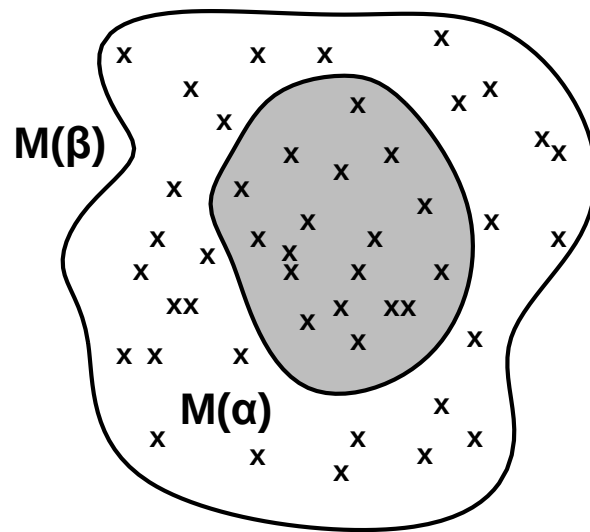
- **蕴含 (entailment)** 即一个语句逻辑上引发另一个语句，其形式化定义为当且仅当在 α 为真的每个模型中 β 也为真，记为：

$$\alpha \models \beta$$

例如， $x + y = 4$ 蕴含 $4 = x + y$

- 利用模型的记号，蕴含的定义也可以写作：

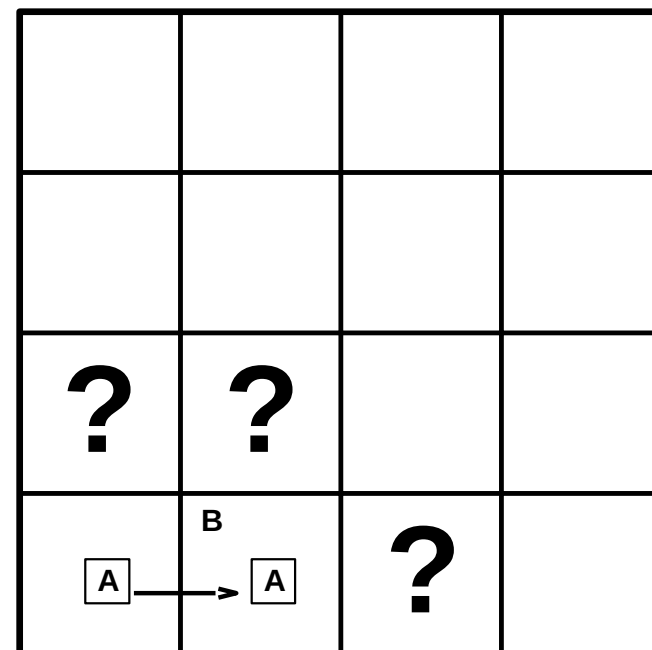
$$\alpha \models \beta \text{ 当且仅当 } M(\alpha) \subseteq M(\beta)$$



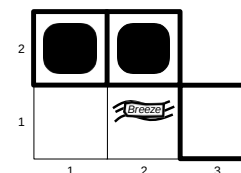
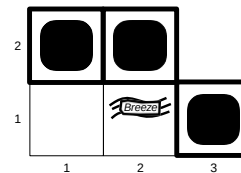
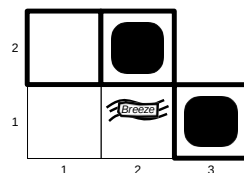
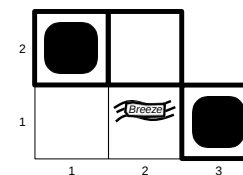
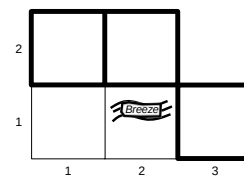
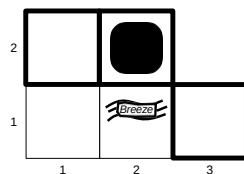
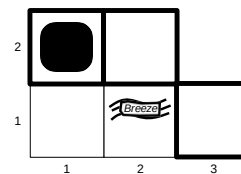
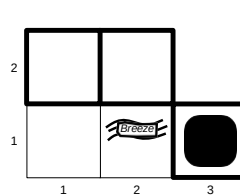
在wumpus世界中的蕴含

智能体在 $[1, 1]$ 中什么都没有探测到，
在 $[1, 2]$ 中探测到微风。

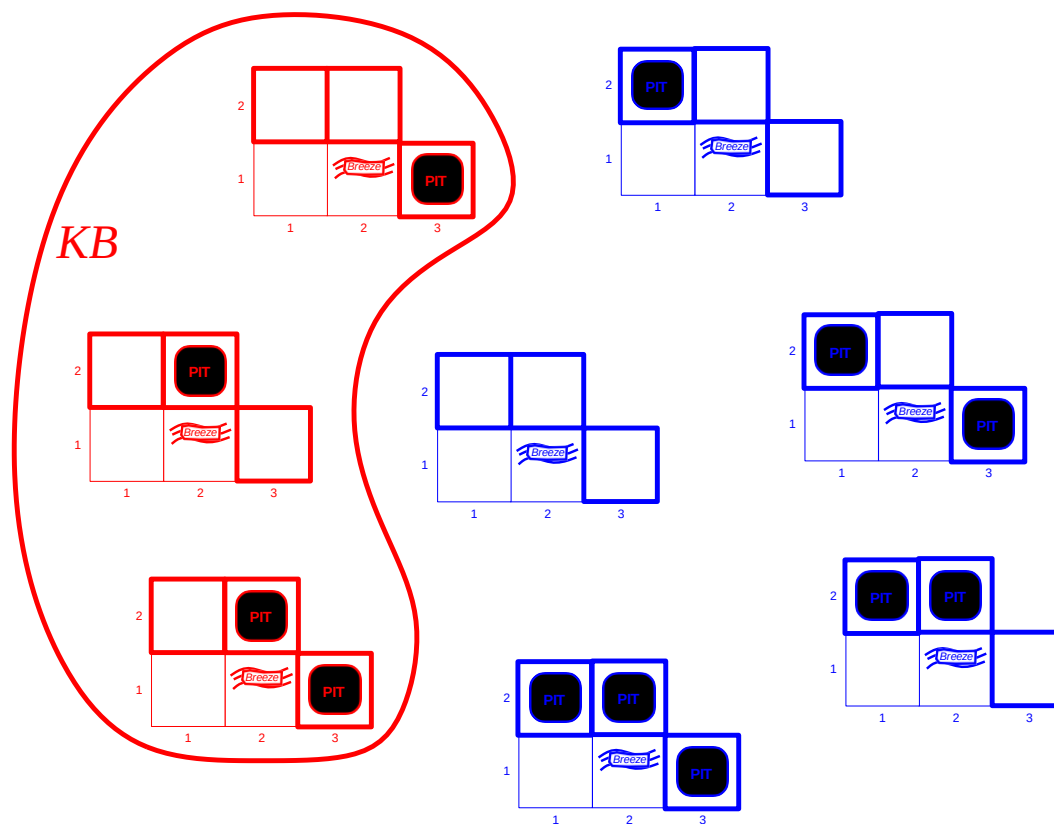
智能体所感兴趣的是相邻的方格 $[2, 1]$ 、
 $[2, 2]$ 和 $[1, 3]$ 是否有无底洞。



在wumpus世界中的蕴含

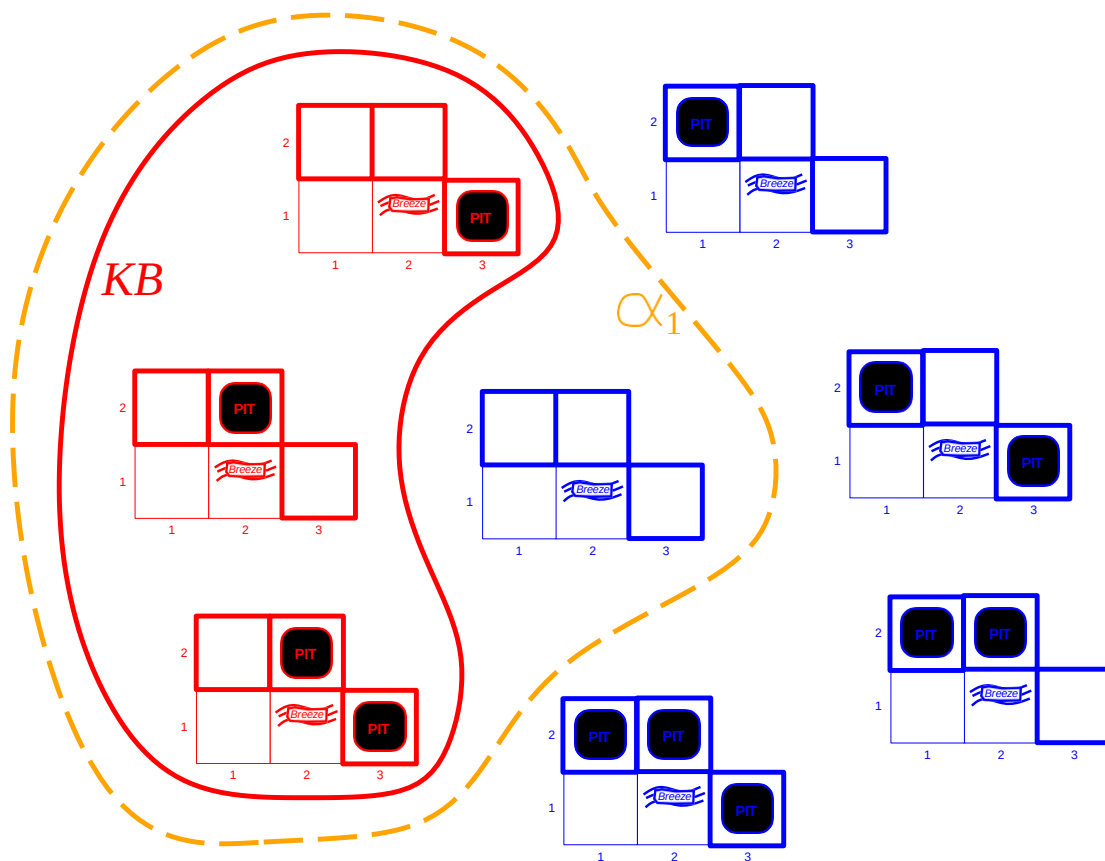


在wumpus世界中的蕴含



KB = wumpus 世界的规则 + 观测

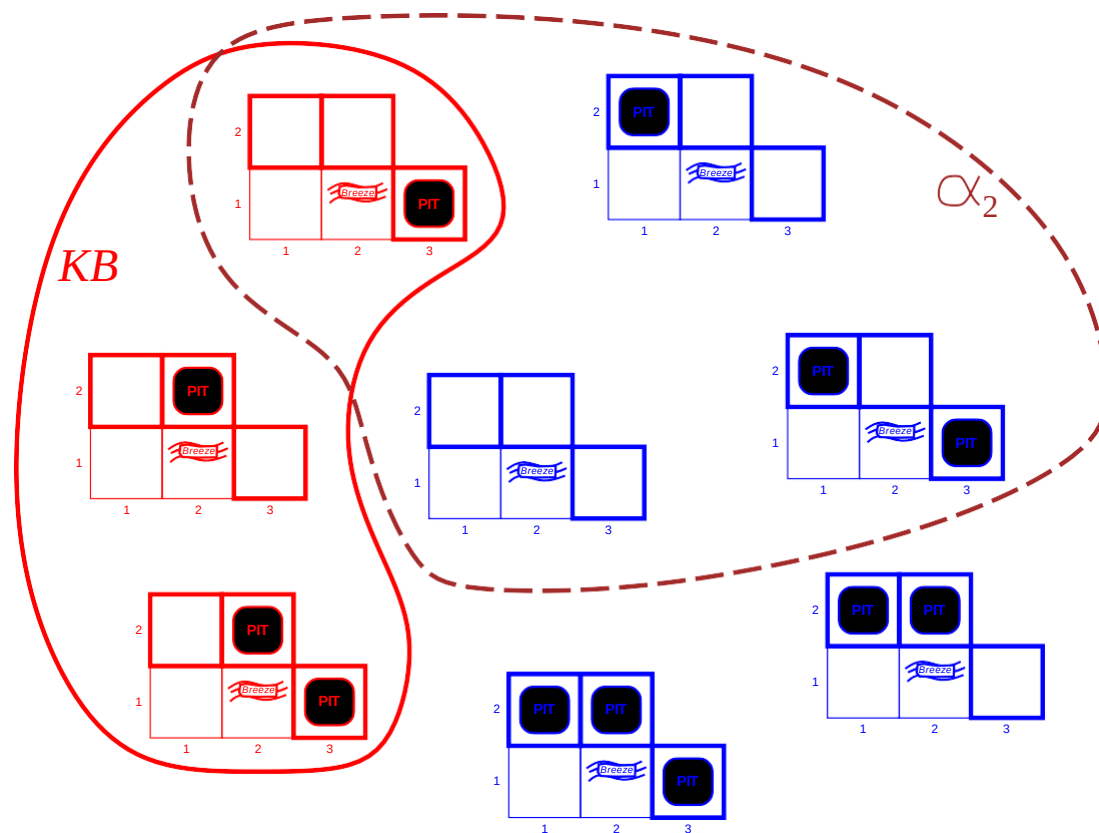
在wumpus世界中的蕴含



KB = wumpus 世界的规则 + 观测

α_1 = “[2,1] 中没有无底洞”, $KB \models \alpha_1$, 可通过模型检验证明

在wumpus世界中的蕴含



KB = wumpus 世界的规则 + 观测

α_2 = “[2,2]中没有无底洞”, KB 不蕴含 α_2

命题逻辑：语法

命题逻辑的**语法**定义合法的语句。**原子语句 (atomic sentence)** 由单个**命题符号 (proposition symbol)** 构成。使用括号和被称作**逻辑联结词 (logical connective)** 的运算符可以将简单语句构造成为**复合语句 (complex sentence)**。常用的联结词有5个：

- $\neg$ (非)。类似 $\neg W_{1,3}$ 这样的语句称为 $W_{1,3}$ 的**否定**。一个**文字**要么是原子语句，即**正文字**，要么是原子语句的否定，即**负文字**。
- $\wedge$ (与)。主要联结词是 $\wedge$ 的语句称为**合取式**，例如 $W_{1,3} \wedge P_{3,1}$ ，其各部分称为**合取子句**。（ $\wedge$ 看起来像是“And”中的“A”。）
- $\vee$ (或)。主要联结词是 $\vee$ 的语句称为**析取式**，例如 $(W_{1,3} \wedge P_{3,1}) \vee W_{2,2}$ ，其各部分为**析取子句**，本例中分别为 $(W_{1,3} \wedge P_{3,1})$ 和 $W_{2,2}$ 。
- $\Rightarrow$ (蕴涵)。如 $(W_{1,3} \wedge P_{3,1}) \Rightarrow W_{2,2}$ 这样的语句称为**蕴涵式 (implication)** 或条件式，其**前提 (premise)** 或**前件 (antecedent)** 是 $(W_{1,3} \wedge P_{3,1})$ ，其**结论 (conclusion)** 或**后件 (consequent)** 是 $W_{2,2}$ 。蕴涵式也被称为**规则 (rule)** 或**if-then** 声明。有时，蕴涵符号在一些书籍中写作 $\supset$ 或 $\rightarrow$ 。
- $\Leftrightarrow$ (当且仅当)。语句 $W_{1,3} \Leftrightarrow \neg W_{2,2}$ 是**双向蕴涵式 (biconditional)**。

命题逻辑中语句的BNF文法以及从高到低排列的运算符优先级：

语句 $\rightarrow$ 原子语句 | 复合语句
原子语句 $\rightarrow True | False | P | Q | R | \dots$
复合语句 $\rightarrow$ (语句)
 | $\neg$ 语句
 | 语句 $\wedge$ 语句
 | 语句 $\vee$ 语句
 | 语句 $\Rightarrow$ 语句
 | 语句 $\Leftrightarrow$ 语句

运算符优先级： $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

语义定义了用于判定特定模型中语句真值的规则。命题逻辑中，模型就是对每个命题符号设定真值，即真 (true) 或假 (false)。命题逻辑的语义必须指定在给定模型下如何计算任一语句的真值。**这是以递归的方式实现的。**所有语句都是由原子语句和5个联结词构建的。

- $\neg P$ 为真，当且仅当在 m 中 P 为假。
- $P \wedge Q$ 为真，当且仅当在 m 中 P 和 Q 都为真。
- $P \vee Q$ 为真，当且仅当在 m 中 P 或 Q 中至少一个为真。
- $P \Rightarrow Q$ 为真，除非在 m 中 P 为真而 Q 为假。
- $P \Leftrightarrow Q$ 为真，当且仅当在 m 中 P 和 Q 都为真或都为假。

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

5个逻辑联结词的真值表。若要使用真值表计算在P为真、Q为假时 $P \vee Q$ 的值，首先在左边找到P为true而Q为false的行（第3行），然后找到该行位于 $P \vee Q$ 列处的值，得到结果true。

可以使用简单的递归过程来计算任一语句的真值, 例如,

$$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = \text{true} \wedge (\text{false} \vee \text{true}) = \text{true} \wedge \text{true} = \text{true}$$

我们已经定义了命题逻辑的语义，现在可以为wumpus世界构建一个知识库了。对于每个位置 $[x, y]$ ，需要用到下列符号：

当 $[x, y]$ 有无底洞， $P_{x, y}$ 为真。

当 wumpus 在 $[x, y]$ ，不论其死活 $W_{x, y}$ 都为真。

当 $[x, y]$ 有微风， $B_{x, y}$ 为真。

当 $[x, y]$ 处有臭味， $S_{x, y}$ 为真。

当智能体位于位置 $[x, y]$ ， $L_{x, y}$ 为真。

命题逻辑：wumpus世界

方格[1, 2]中没有无底洞的推导过程，我们用 R_i 代表每个语句，以便推导。

- 方格[1, 1]中没有无底洞： $R_1 : \neg P_{1,1}$
- 一个方格有微风，当且仅当其相邻方格中有无底洞。必须对每个方格都进行这样的表示，在此我们只写出相关方格的表示：

$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

- 上述语句为wumpus世界的规则。
我们现在为智能体在这个特定世界中已访问过的前两个方格引入**观测**，以形成右图所示的情形：

$$R_4 : \neg B_{1,1}$$

$$R_5 : B_{2,1}$$

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

(b)

命题逻辑：通过真值表推断

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	false	true	true	true	true	true	<u>true</u>
false	true	false	false	false	false	true	true	true	true	true	true	<u>true</u>
false	true	false	false	false	false	false	true	true	true	true	true	<u>true</u>
false	true	false	false	true	false	false	true	false	false	true	true	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
true	true	true	true	true	true	true	false	true	true	false	true	false

根据wumpus世界的知识库 KB 构建的真值表

通过查找真值表的方式得出 KB 蕴含 “方格[1, 2]中没有无底洞” 这一语句

命题定理证明：逻辑等价

逻辑等价 (logical equivalence)：如果两个语句在相同的模型集合中都为真，则这两个语句逻辑等价，可以写作： $\alpha \equiv \beta$

等价的另一种定义为任意两条语句是等价的，当且仅当它们互相蕴含：

$$\alpha \equiv \beta \text{ 当且仅当 } \alpha \models \beta \text{ 且 } \beta \models \alpha$$

标准的逻辑等价： $(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$ $\wedge$ 的交换律

$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \vee \text{ 的交换律}$$

$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \wedge \text{ 的结合律}$$

$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \vee \text{ 的结合律}$$

$$\neg(\neg\alpha) \equiv \alpha \quad \text{双重否定律}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{假言易位}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{蕴涵消去}$$

$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{等价消去}$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{德摩根律}$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{德摩根律}$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \wedge \text{ 对 } \vee \text{ 的分配律}$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \vee \text{ 对 } \wedge \text{ 的分配律}$$

命题定理证明：有效性和可满足性

- **有效性 (validity)** : 如果一条语句在所有模型中都为真, 则这条语句是有效的。

例如 $True, A \vee \neg A, A \Rightarrow A, (A \wedge (A \Rightarrow B)) \Rightarrow B$

蕴含的定义可以推导出演绎定理:

$KB \models a$ 当且仅当 $(KB \Rightarrow a)$ 是有效的

命题定理证明：有效性和可满足性

- **有效性 (validity)** : 如果一条语句在所有模型中都为真, 则这条语句是有效的。

例如 $True, A \vee \neg A, A \Rightarrow A, (A \wedge (A \Rightarrow B)) \Rightarrow B$

蕴含的定义可以推导出演绎定理:

$KB \models a$ 当且仅当 $(KB \Rightarrow a)$ 是有效的

- **可满足性 (satisfiability)** : 如果一条语句在某些模型中为真或能够被满足, 则这条语句是可满足的。

如果一条语句没有模型使其为真, 则这条语句是不可满足的。

例如, $A \wedge \neg A$

可满足可以推导出归谬法:

$KB \models a$ 当且仅当 $(KB \wedge \neg a)$ 是不可满足的

- **单调性**: 它表明蕴含的语句集只能随着信息被加入知识库而增长。

如果 $KB \models a$, 则 $KB \wedge \beta \models a$

- **肯定前件** (Modus Ponens, mode that affirms的拉丁语) , 写作

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

- **合取消去** (and-elimination) , 即可以从一个合取式推导出任一合取子句:

$$\frac{\alpha \wedge \beta}{\alpha}$$

- 所有逻辑等价都可以用作推断规则。例如, 等价消去可以产生两条推断规则:

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta)(\beta \Rightarrow \alpha)} \text{ 和 } \frac{(\alpha \Rightarrow \beta)(\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

命题逻辑：wumpus世界

方格[1, 2]中没有无底洞的推导过程，我们用 R_i 代表每个语句，以便推导。

- 方格[1, 1]中没有无底洞： $R_1 : \neg P_{1,1}$
- 一个方格有微风，当且仅当其相邻方格中有无底洞。必须对每个方格都进行这样的表示，在此我们只写出相关方格的表示：

$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

- 上述语句为wumpus世界的规则。
我们现在为智能体在这个特定世界中已访问过的前两个方格引入**观测**，以形成右图所示的情形：

$$R_4 : \neg B_{1,1}$$

$$R_5 : B_{2,1}$$

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

(b)

命题定理证明：推断与证明

- 推断规则和等价关系应用于wumpus世界。从含有R1到R5的知识库开始，演示如何证明[1, 2]中没有无底洞。首先给出 $R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

(1) 对 R_2 使用等价消去，得到

$$R_6 : (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

(2) 对 R_6 使用合取消去，得到

$$R_7 : ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

(3) 假言易位逻辑等价关系得到

$$R_8 : (\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1}))$$

(4) 对 R_8 和感知 $R4$ (即 $\neg B_{1,1}$) 使用肯定前件，得到

$$R_9 : \neg(P_{1,2} \vee P_{2,1})$$

(5) 使用德摩根律，得到结论

$$R_{10} : \neg P_{1,2} \wedge \neg P_{2,1}$$

也就是，[1, 2]和[2, 1]都没有无底洞。

应用第3章的任意搜索算法都可以找到构成这种证明的一系列步骤。
只需要定义如下的证明问题：

- **初始状态 (Initial State)**：最初的知识库。
- **动作 (Actions)**：动作的集合，包含所有推断规则应用于所有符合上半部分推断规则的语句。
- **结果 (Result) 或者转移模型**：将推断规则下半部分的语句实例加入知识库。
- **目标 (Goal)**：目标是含有我们想要证明的语句的状态。

命题定理证明：通过归结证明

单元归结 (unit resolution) 规则

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \quad m}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k}$$

单元归结规则可以推广为**全归结规则**

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \quad m_1 \vee \cdots \vee m_n}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

归结使用两个子句并产生一个新的子句，该新子句包含除一对互补文字以外的原始子句的所有文字，例如

$$\frac{P_{1,1} \vee P_{3,1}, \quad \neg P_{1,1} \vee \neg P_{2,2}}{P_{3,1} \vee \neg P_{2,2}}$$

一次只能归结一对互补文字，例如

$$\frac{P \vee \neg Q \vee R, \quad \neg P \vee Q}{\neg Q \vee Q \vee R}$$

命题定理证明：通过归结证明

形式为子句合取式的语句被称为**合取范式 (conjunctive normal form) 或CNF**。把语句转换为CNF的过程：

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

1. 消去 $\Leftrightarrow$, 替换 $a \Leftrightarrow b$ 为 $(a \Rightarrow b) \wedge (b \Rightarrow a)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

2. 消去 $\Rightarrow$, 替换 $a \Rightarrow b$ 为 $\neg a \vee b$.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. 将 $\neg$ 内移, 通过使用德摩根律:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. 应用分配律:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

命题定理证明：通过归结证明

function PL-RESOLUTION(KB, a) **returns** true或false

inputs: KB , 知识库, 一个命题逻辑中的语句

a , 查询, 一个命题逻辑中的语句

$clauses \leftarrow KB \wedge \neg a$ 的CNF表示的子句集合

$new \leftarrow \{\}$

while true **do**

for each 子句对 C_i, C_j **in** $clauses$ **do**

$resolvents \leftarrow \text{PL-RESOLVE}(C_i, C_j)$

if $resolvents$ 包含空子句 **then return** true

$new \leftarrow new \cup resolvents$

if $new \subseteq clauses$ **then return** false

$clauses \leftarrow clauses \cup new$

简单的命题逻辑归结算法。PL-Resolve返回对其两个输入进行归结得到的所有可能子句集合。

命题定理证明：通过归结证明

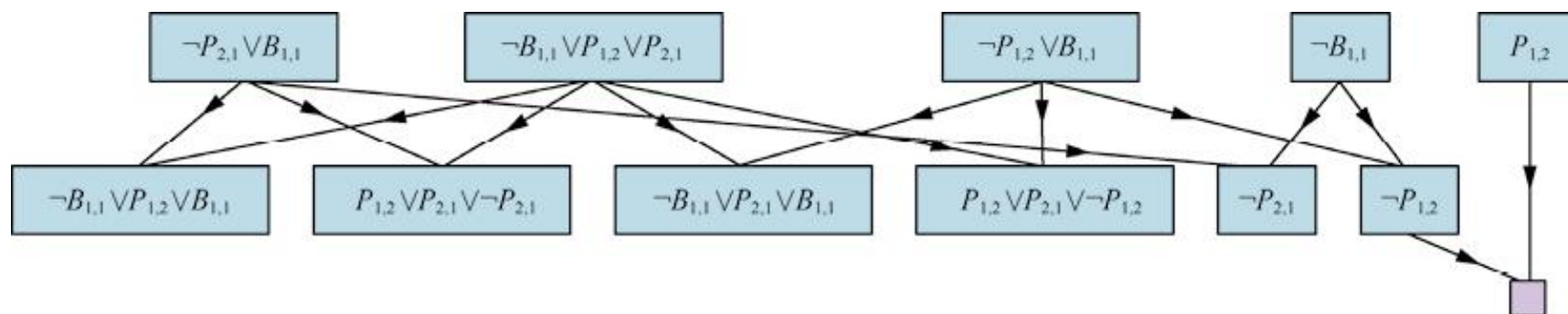
相关的知识库： $KB = R_2 \wedge R_4 = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$

证明： $\neg P_{1,2}$

命题定理证明：通过归结证明

相关的知识库： $KB = R_2 \wedge R_4 = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$

证明： $\neg P_{1,2}$



对wumpus世界的一个简单推断部分运用PL-Resolution来证明查询。

知识点回顾

◆ 第十二章：不确定性的量化

- 贝叶斯法则
- 朴素贝叶斯模型

- 为了让智能体能够表示和使用概率信息，我们需要一种形式语言。
- **样本空间（记为 Ω ）**：所有可能世界的集合。
 - 这些可能世界是互斥的和可穷举的
- 一个完全指定的概率模型为每个可能世界 ω 赋予一个数值概率 $P(\omega)$ 。
- 概率论基本公理规定每个可能世界具有一个0到1之间的概率，并且样本空间 Ω 中的可能世界的总概率为1：
$$0 \leq P(\omega) \leq 1 \text{ 对任一 } \omega \text{ 且 } \omega \in \Omega$$
- **先验概率**：无任何其他信息下命题的概率。已经观测到某些**证据 (evidence)** 的情况下的概率称为**后验概率**。

基本概率记号

- **因子化表示:** 可能世界由 “变量/值” 对的集合来表示。
 - 概率论中的变量被称为随机变量，它们的名字首字母大写。
- 有时要讨论一个随机变量的所有可能的值，我们可以写作：

$$\begin{aligned}P(\textit{Weather} = \textit{sun}) &= 0.6 \\P(\textit{Weather} = \textit{rain}) &= 0.1 \\P(\textit{Weather} = \textit{cloud}) &= 0.29 \\P(\textit{Weather} = \textit{snow}) &= 0.01 ,\end{aligned}$$

- 也可以简写为：

$$P(\textit{Weather}) = (0.6, 0.1, 0.29, 0.01),$$

- P 为随机变量 $\textit{Weather}$ 定义了一个**概率分布**。
- 除单变量的分布外，我们还需要多变量分布的记号。这里使用逗号来分隔不同变量。例如， $P(\textit{Weather}, \textit{Cavity})$ 表示 $\textit{Weather}$ 和 $\textit{Cavity}$ 所有组合值的概率。这是一个 4×2 的概率表，称为 $\textit{Weather}$ 和 $\textit{Cavity}$ 的**联合概率分布**。

事件的概率

- 一个事件是样本空间 Ω 的任一子集。类似的，在逻辑学中，一组可能世界的集合对应形式语言中的一个命题。
- 一个事件的概率是其可能世界的概率之和：
 - $P(A) = \sum_{\omega \in A} P(\omega)$
- 给定所有变量的联合分布，能够计算任一事件的概率
 - 气温高并且是晴天的概率？
 - 气温高的概率？
 - 气温高或者是非雾天的概率？

$P(T, W)$

		Temperature	
		hot	cold
Weather	sun	0.45	0.15
	rain	0.02	0.08
	fog	0.03	0.27
	meteor	0.00	0.00

边缘概率

- 边缘概率分布是联合概率分布表消去一些变量后的子表。
- 边缘化 (求和消元): 通过求和来消去一个维度。

$$P(X=x) = \sum_y P(X=x, Y=y)$$

		Temperature		
		hot	cold	
Weather	sun	0.45	0.15	0.60
	rain	0.02	0.08	0.10
	fog	0.03	0.27	0.30
	meteor	0.00	0.00	0.00
		0.50	0.50	

$P(W)$

$P(T)$

- **条件概率:** 给定证据 (evidence) 已经发生时, 新事件的信念度。即, 给定条件变量的取值之后目标变量的概率分布。

- 给定命题 b , 命题 a 的概率:

$$P(a|b) = \frac{P(a \wedge b)}{P(b)}$$

- 也可以写成**乘积法则**形式:

$$P(a \wedge b) = P(a|b)P(b) .$$

- 例如, 给定第一个骰子是5的前提下两个骰子点数相同的概率:

$$P(doubles|Die_1 = 5) = \frac{P(doubles \wedge Die_1 = 5)}{P(Die_1 = 5)} .$$

- 通过重复应用乘积法则，联合概率分布可以重写为条件概率分布的乘积：

$$P(x_1, x_2, x_3) = P(x_3 \mid x_1, x_2) P(x_1, x_2) = P(x_3 \mid x_1, x_2) P(x_2 \mid x_1) P(x_1)$$

$$P(x_1, x_2, \dots, x_n) = \prod_i P(x_i \mid x_1, \dots, x_{i-1})$$

使用完全联合分布进行推断

从联合概率分布开始:

	<i>toothache</i>		$\neg$ <i>toothache</i>	
	<i>catch</i>	$\neg$ <i>catch</i>	<i>catch</i>	$\neg$ <i>catch</i>
<i>cavity</i>	.108	.012	.072	.008
$\neg$ <i>cavity</i>	.016	.064	.144	.576

对任一事件（命题） φ , 对事件（命题）为真的可能世界的概率相加:

$$P(\varphi) = \sum_{\omega: \omega \models \varphi} P(\omega)$$

使用完全联合分布进行推断

从联合概率分布开始:

	<i>toothache</i>		$\neg$ <i>toothache</i>	
	<i>catch</i>	$\neg$ <i>catch</i>	<i>catch</i>	$\neg$ <i>catch</i>
<i>cavity</i>	.108	.012	.072	.008
$\neg$ <i>cavity</i>	.016	.064	.144	.576

对任一事件（命题） φ , 对事件（命题）为真的可能世界的概率相加:

$$P(\varphi) = \sum_{\omega: \omega \models \varphi} P(\omega)$$

$$P(\text{toothache}) = 0.108 + 0.012 + 0.016 + 0.064 = 0.2$$

使用完全联合分布进行推断

从联合概率分布开始:

	<i>toothache</i>		$\neg$ <i>toothache</i>	
	<i>catch</i>	$\neg$ <i>catch</i>	<i>catch</i>	$\neg$ <i>catch</i>
<i>cavity</i>	.108	.012	.072	.008
$\neg$ <i>cavity</i>	.016	.064	.144	.576

对任一事件（命题） φ , 对事件（命题）为真的可能世界的概率相加:

$$P(\varphi) = \sum_{\omega: \omega \models \varphi} P(\omega)$$

$$P(\text{cavity} \vee \text{toothache}) =$$

$$0.108 + 0.012 + 0.072 + 0.008 + 0.016 + 0.064 = 0.28$$

使用完全联合分布进行推断

从联合概率分布开始:

	<i>toothache</i>		$\neg$ <i>toothache</i>	
	<i>catch</i>	$\neg$ <i>catch</i>	<i>catch</i>	$\neg$ <i>catch</i>
<i>cavity</i>	.108	.012	.072	.008
$\neg$ <i>cavity</i>	.016	.064	.144	.576

也可以计算条件概率:

$$\begin{aligned} P(\neg \text{cavity} | \text{toothache}) &= \frac{P(\neg \text{cavity} \wedge \text{toothache})}{P(\text{toothache})} \\ &= \frac{0.016 + 0.064}{0.108 + 0.012 + 0.016 + 0.064} = 0.4 \end{aligned}$$

	<i>toothache</i>		<i>toothache</i>	
	<i>catch</i>	$\neg$ <i>catch</i>	<i>catch</i>	$\neg$ <i>catch</i>
<i>cavity</i>	.108	.012	.072	.008
$\neg$ <i>cavity</i>	.016	.064	.144	.576

$P(\text{toothache})$ 可以被看作 $P(\text{Cavity} | \text{toothache})$ 的归一化常量

$$\begin{aligned}
 P(\text{Cavity} | \text{toothache}) &= \alpha P(\text{Cavity}, \text{toothache}) \\
 &= \alpha [P(\text{Cavity}, \text{toothache}, \text{catch}) + P(\text{Cavity}, \text{toothache}, \neg \text{catch})] \\
 &= \alpha [(0.108, 0.016) + (0.012, 0.064)] \\
 &= \alpha (0.12, 0.08) = (0.6, 0.4)
 \end{aligned}$$

令 E 表示证据变量列表， e 表示它们的观测值列表， Y 表示剩余未观测变量。
 $P(X | e)$ 可以计算为：

$$P(X | e) = \alpha P(X, e) = \alpha \sum_y P(X, e, y)$$

其中求和是针对所有可能的 y （也就是未观测变量 Y 的值的的所有可能组合）。

贝叶斯法则及其应用

- 乘积法则推可以写成两种形式:

$$P(a \wedge b) = P(a|b)P(b) \quad \text{和} \quad P(a \wedge b) = P(b|a)P(a) .$$

- 联立两式右侧, 除以 $P(a)$, 我们可以得到贝叶斯法则:

$$P(b|a) = \frac{P(a|b)P(b)}{P(a)} .$$

贝叶斯法则及其应用

- 乘积法则推可以写成两种形式:

$$P(a \wedge b) = P(a|b)P(b) \quad \text{和} \quad P(a \wedge b) = P(b|a)P(a) .$$

- 联立两式右侧, 除以 $P(a)$, 我们可以得到贝叶斯法则:

$$P(b|a) = \frac{P(a|b)P(b)}{P(a)} .$$

- 通常, 我们把一些未知**原因** (cause) 的**结果** (effect) 视为证据, 并想要确定这个原因。在这种情况下, 贝叶斯法则变为了:

$$P(\text{cause}|\text{effect}) = \frac{P(\text{effect}|\text{cause})P(\text{cause})}{P(\text{effect})}$$

- 条件概率 $P(\text{effect}|\text{cause})$ 量化**因果**方向上的关系, 而 $P(\text{cause}|\text{effect})$ 描述**诊断**方向上的关系。

- 例如，医生知道脑膜炎在70%的情况下会导致病人颈部僵硬。医生也知道一些先验概率：病人患有脑膜炎的先验概率是1/50000，而病人颈部僵硬的先验概率是1%。令s表示病人颈部僵硬的命题，m表示病人患有脑膜炎的命题，我们有：

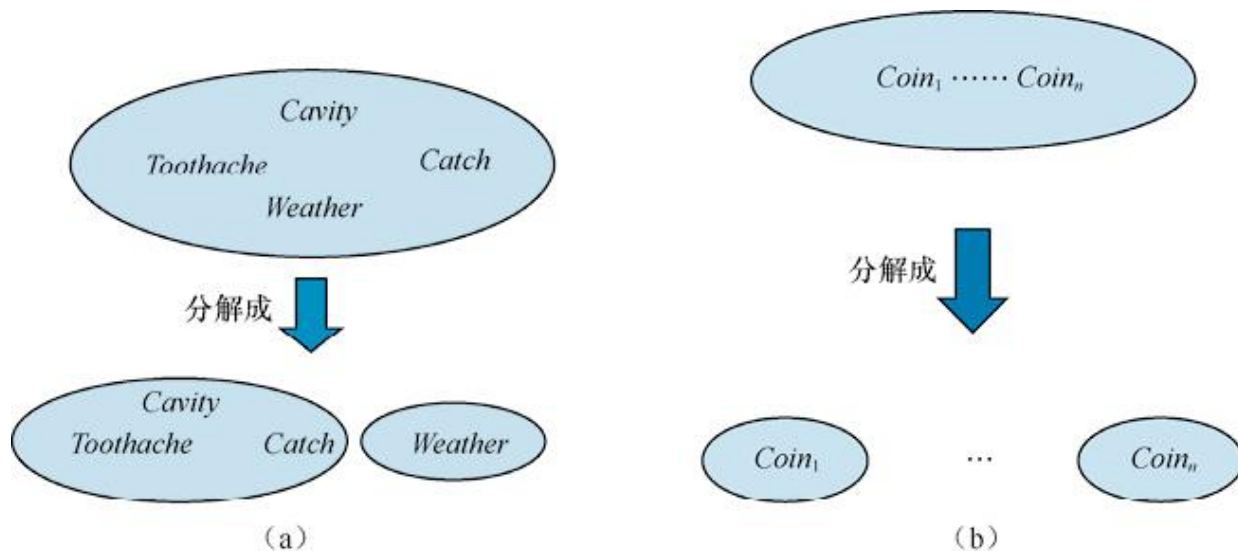
$$P(s|m) = 0.7$$

$$P(m) = 1/50000$$

$$P(s) = 0.01$$

$$P(m|s) = \frac{P(s|m)P(m)}{P(s)} = \frac{0.7 \times 1/50000}{0.01} = 0.0014$$

- 也就是说，我们预计只有0.14%的颈部僵硬的病人患有脑膜炎。注意，尽管脑膜炎显著地表现为颈部僵硬（概率为0.7），但颈部僵硬的病人患有脑膜炎的概率仍然很小。这是因为（任何原因导致的）颈部僵硬的先验概率比患有脑膜炎的先验概率高得多。



使用绝对独立性将一个大的联合分布分解成小分布的两个例子。(a) 天气和牙齿问题是独立的。(b) 抛硬币是独立的。

- 命题a和b独立可以写作:

$$P(a|b) = P(a) \text{ 或 } P(b|a) = P(b) \text{ 或 } P(a \wedge b) = P(a)P(b).$$

- 例如, 使用乘积法则可以得到:

- $P(\text{toothache}, \text{catch}, \text{cavity})$

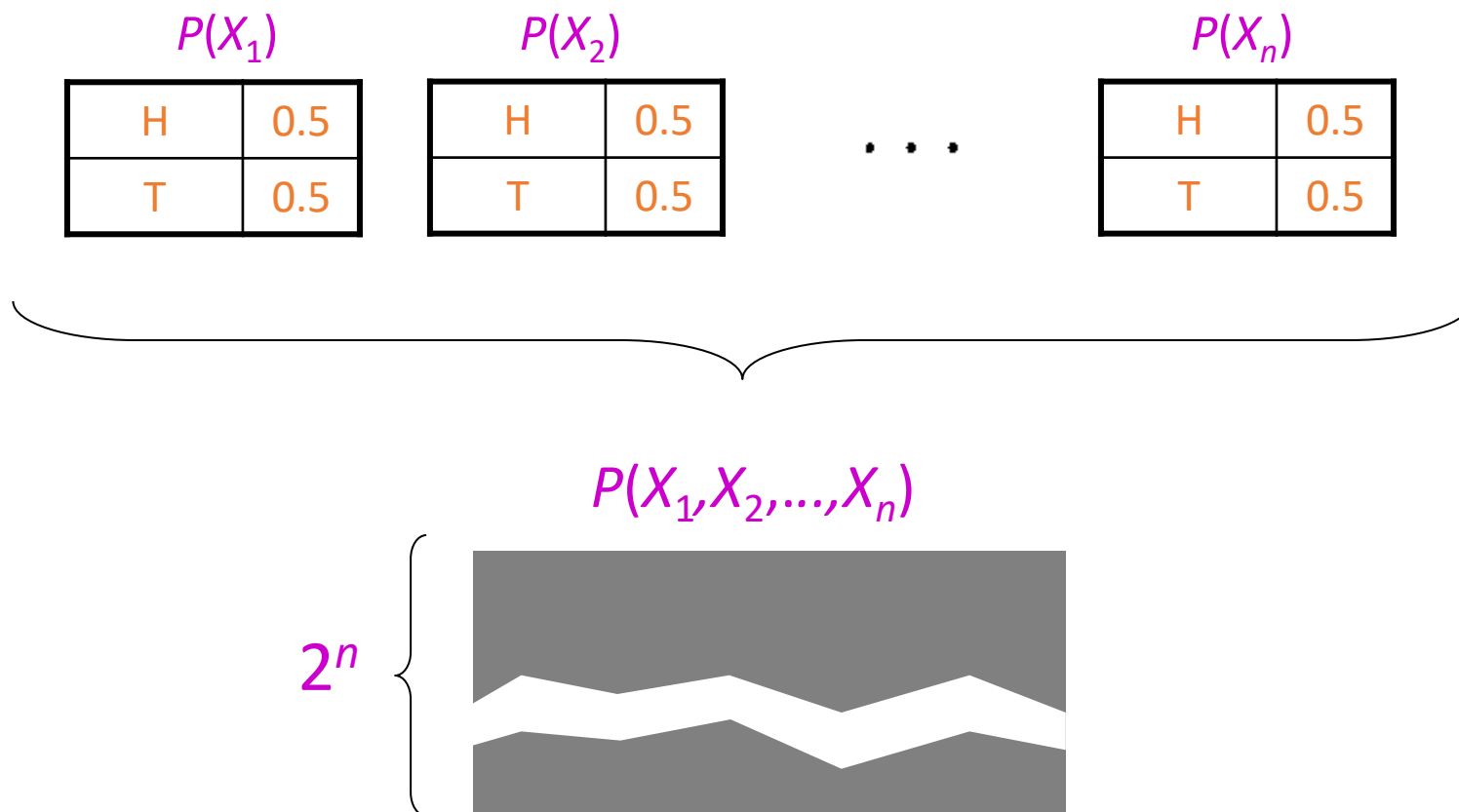
$$= P(\text{cloud} | \text{toothache}, \text{catch}, \text{cavity}) P(\text{toothache}, \text{catch}, \text{cavity}).$$

- 独立性: $P(\text{cloud} | \text{toothache}, \text{catch}, \text{cavity}) = P(\text{cloud}).$

- $P(\text{toothache}, \text{catch}, \text{cavity}, \text{cloud}) = P(\text{cloud})P(\text{toothache}, \text{catch}, \text{cavity})$

独立性

- n 次独立的掷硬币:



条件独立性

- 条件独立性 是建模不确定环境最基本也是最稳健的方式。
- 给定 z ， x 条件独立于 y ，当且仅当：

$$\forall x, y, z \quad P(x \mid y, z) = P(x \mid z)$$

或者，等价的，当且仅当

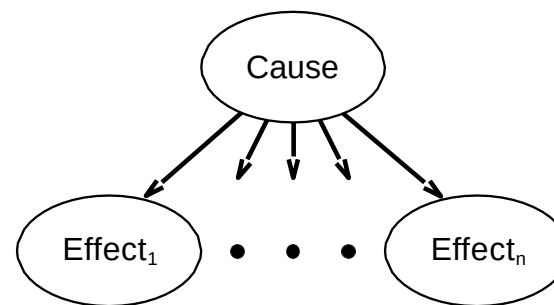
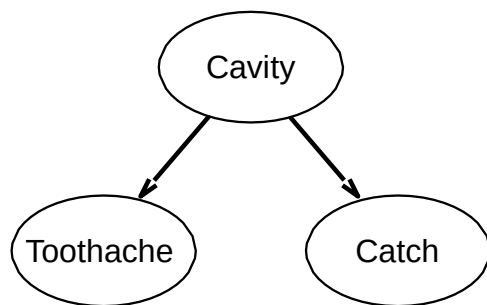
$$\forall x, y, z \quad P(x, y \mid z) = P(x \mid z) P(y \mid z)$$

贝叶斯法则与条件独立性

- 利用贝叶斯法则和条件独立性:

$$\begin{aligned} P(Cavity | toothache \wedge catch) \\ &= a P(toothache \wedge catch | Cavity) P(Cavity) \\ &= a P(toothache | Cavity) P(catch | Cavity) P(Cavity) \end{aligned}$$

- 这是朴素贝叶斯模型的一个例子:



- 参数总量与 n 是线性关系。

朴素贝叶斯模型

- 单个原因直接影响许多结果，给定原因时，所有这些结果都是条件独立的。此时，完全联合分布可以写作：

$$P(\text{Cause}, \text{Effect}_1, \dots, \text{Effect}_n) = P(\text{Cause}) \prod_i P(\text{Effect}_i | \text{Cause})$$

- 这样的概率分布叫作朴素贝叶斯（naive Bayes）模型。“朴素”是因为它经常（作为一种简化假设）用于在给定原因变量时，“结果”变量不是严格独立的情况。
- 考虑观测到的结果 $\mathbf{E}=\mathbf{e}$, 剩余结果变量 $\mathbf{Y}$ 是未观测的：

$$P(\text{Cause} | \mathbf{e}) = \alpha \sum_{\mathbf{y}} P(\text{Cause}, \mathbf{e}, \mathbf{y})$$

- 根据朴素贝叶斯，我们有

$$\begin{aligned} P(\text{Cause} | \mathbf{e}) &= \alpha \sum_{\mathbf{y}} P(\text{Cause}) P(\mathbf{y} | \text{Cause}) \left(\prod_j P(e_j | \text{Cause}) \right) \\ &= \alpha P(\text{Cause}) \left(\prod_j P(e_j | \text{Cause}) \right) \sum_{\mathbf{y}} P(\mathbf{y} | \text{Cause}) \\ &= \alpha P(\text{Cause}) \prod_j P(e_j | \text{Cause}) \end{aligned}$$

知识点回顾

◆ 第十三章：概率推理

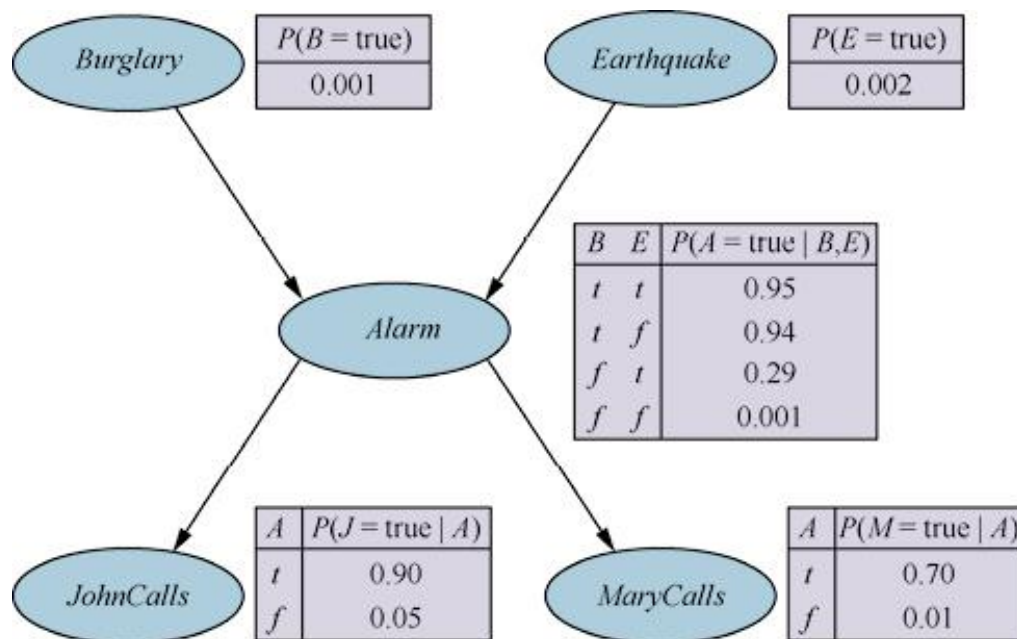
- 不确定域的知识表示
- 贝叶斯网络的语义
- 贝叶斯网络中的精确推断

不确定域的知识表示

- **贝叶斯网络**：一种用局部分布（条件概率分布）来描述联合分布的模型，也称作概率图模型。
- 贝叶斯网络是一个有向图，其中每个节点用定量的概率信息标记。
完整的描述如下：
 - 一组节点，每个节点对应一个随机变量
 - 一个有向无环图(链路 $\approx$ “直接影响”)
 - 给定其父节点，每个节点关联一个条件概率分布表：
 $P(X_i | Parents(X_i))$
- 网络的拓扑（节点和链路的集合）以精确简洁的方式指定了在域中成立的条件独立性关系。
- 一旦我们确定了贝叶斯网络的拓扑结构，就只需要以给定其父节点的条件分布的形式来指定每个变量的局部概率信息。
- **贝叶斯网络 = 拓扑（图）+ 局部条件概率。**

不确定域的知识表示

范例



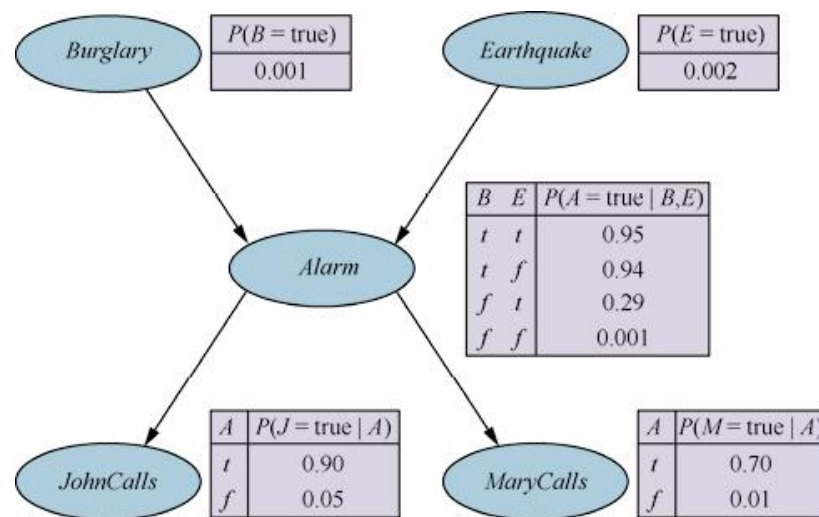
一个典型的贝叶斯网络，所示为其拓扑结构和条件概率表。在条件概率表中，字母B、E、A、J和M分别代表Burglary、Earthquake、Alarm、JohnCalls和MaryCalls。

全局语义定义完全联合分布为局部条件分布的乘积:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(X_i))$$

例如, $P(j \wedge m \wedge a \wedge \neg b \wedge \neg e)$

=



全局语义定义完全联合分布为局部条件分布的乘积:

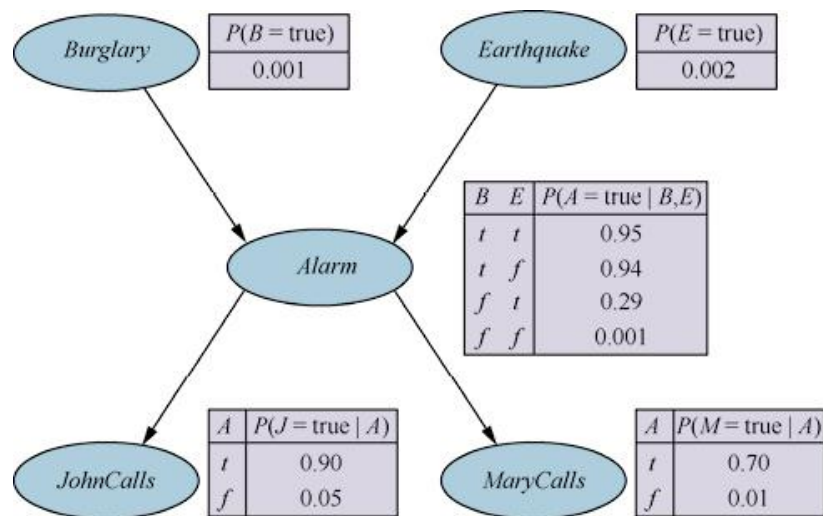
$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(X_i))$$

例如, $P(j \wedge m \wedge a \wedge \neg b \wedge \neg e)$

$$= P(j|a)P(m|a)P(a|\neg b, \neg e)P(\neg b)P(\neg e)$$

$$= 0.9 \times 0.7 \times 0.001 \times 0.999 \times 0.998$$

$$\approx 0.00063$$



贝叶斯网络的语义

贝叶斯网络中的条件独立性关系

- 将贝叶斯网络的全局语义

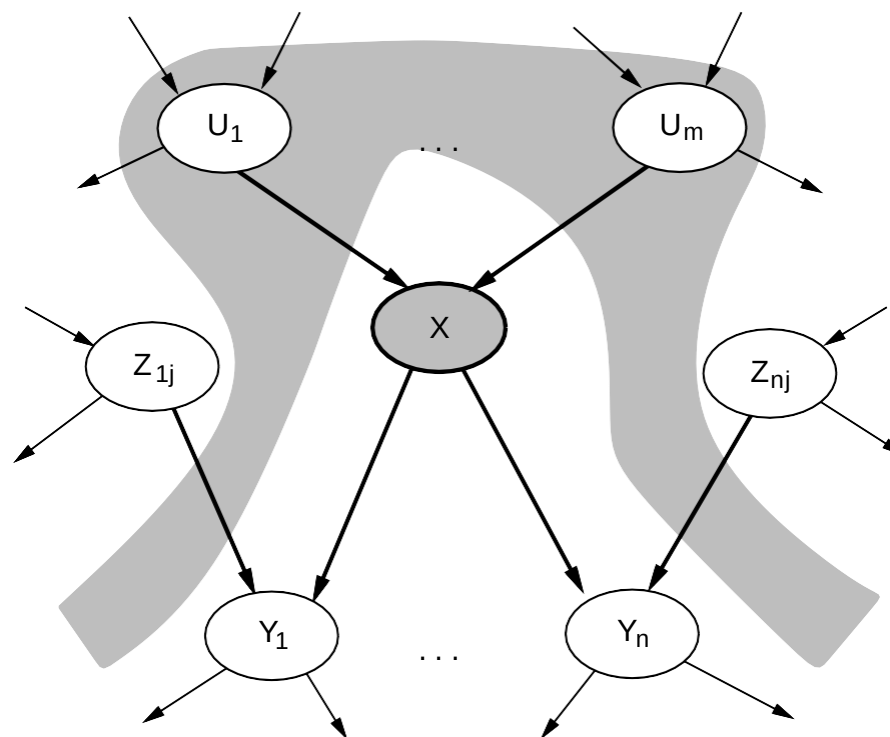
$$P(X_1, \dots, X_n) = \prod_i P(X_i \mid \text{Parents}(X_i))$$

与链式法则恒等式进行比较

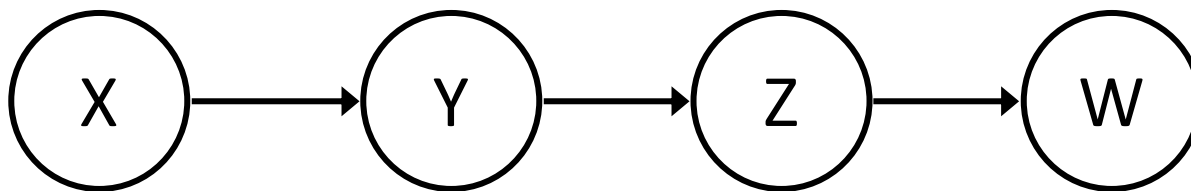
$$P(X_1, \dots, X_n) = \prod_i P(X_i \mid X_1, \dots, X_{i-1})$$

- 假设 (不失一般性) $X_1, \dots, X_n$ 按照图中的拓扑顺序进行存储(即, 父亲节点要先于孩子节点), 因此 $\text{Parents}(X_i) \subseteq X_1, \dots, X_{i-1}$
- 因此贝叶斯网络断言了条件独立性 $P(X_i \mid X_1, \dots, X_{i-1}) = P(X_i \mid \text{Parents}(X_i))$

局部语义: 给定其父结点, 每个变量条件独立于它的非子孙节点



贝叶斯网络的条件独立性



- 条件独立性假设可以从链式法则的简化表示直接推导而来:

$$P(x, y, z, w) = P(x)P(y|x)P(z|x, y)P(w|x, y, z)$$

$$P(x, y, z, w) = P(x)P(y|x)P(z|y)P(w|z)$$

- 蕴含的条件独立性

$$X \perp\!\!\!\perp Z|Y \quad W \perp\!\!\!\perp \{X, Y\}|Z$$

- 其他隐含的条件独立性假设?

$$W \perp\!\!\!\perp X|Y$$

贝叶斯网络的条件独立性- D分离

- 查询: $X_i \perp\!\!\!\perp X_j | \{X_{k_1}, \dots, X_{k_n}\}$
- 检查 X_i 和 X_j 之间的所有路径
 - 如果一条或者多条路径是激活的, 那么条件独立性就不成立
- 否则 (即, 所有的路径都不激活), 条件独立性就成立

$$X_i \not\perp\!\!\!\perp X_j | \{X_{k_1}, \dots, X_{k_n}\}$$

$$X_i \perp\!\!\!\perp X_j | \{X_{k_1}, \dots, X_{k_n}\}$$

贝叶斯网络的条件独立性- D分离

激活与非激活路径

- 如果路径中每一个三元组都是激活的，则该路径是激活的:

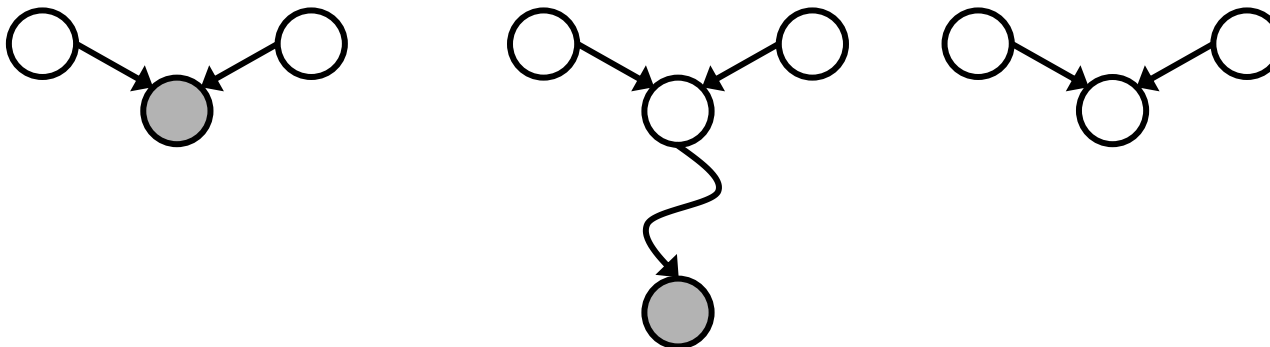
- 因果链 $A \rightarrow B \rightarrow C$



- 共同原因 $A \leftarrow B \rightarrow C$



- 共同结果 (V型结构) $A \rightarrow B \leftarrow C$

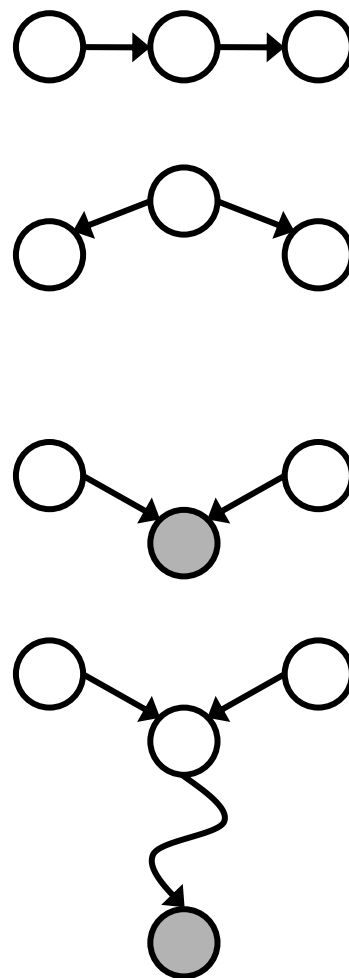


- 如果路径中某一个三元组是非激活的，则该路径就是非激活的

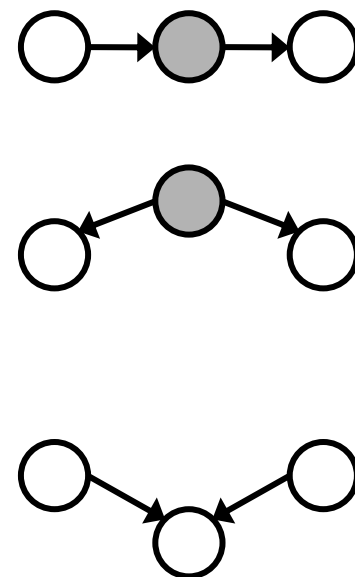
贝叶斯网络的条件独立性- D分离

- 如果路径中每一个三元组都是激活的，则该路径是激活的：
 - 因果链 $A \rightarrow B \rightarrow C$
 - B未被观察，A和C不相互独立
 - 否则，A和C相互独立
 - 共同原因 $A \leftarrow B \rightarrow C$
 - B未被观察，A和C不相互独立
 - 否则，A和C相互独立
 - 共同结果 (V型结构) $A \rightarrow B \leftarrow C$
 - B或者它的孙子节点未被观察，A和C相互独立
 - 否则，A和C不相互独立
- 如果路径中某一个三元组是非激活的，则该路径就是非激活的

激活三元组



非激活三元组



贝叶斯网络的条件独立性- D分离

- 一般形式的问题：在一个给定的贝叶斯网络中, 给定证据变量, 两个变量之间是否独立?
- D-分离：在贝叶斯网路中寻找关联路径, 并基于路径做独立性判断。
- 任何复杂的路径都可以分解为上述三个典型类型的三元组

贝叶斯网络的条件独立性- D分离

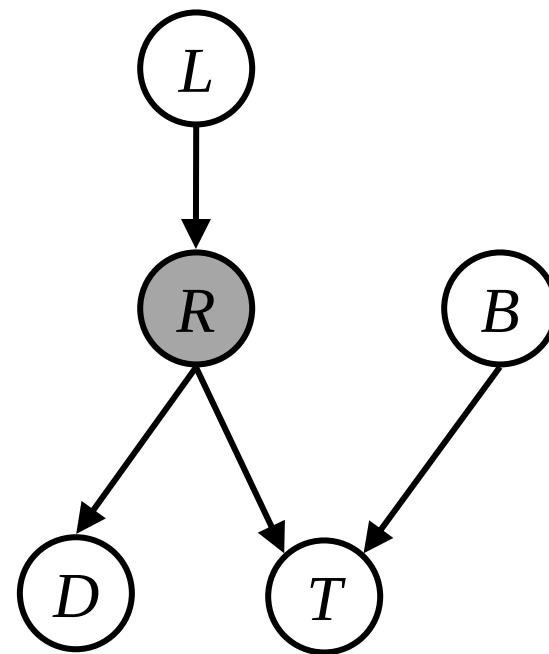
- 给定R, L是否独立于B?
- 路径: $L \rightarrow R^* \rightarrow T \leftarrow B$
- 三元组: $L \rightarrow R^* \rightarrow T$; $R^* \rightarrow T \leftarrow B$

$L \perp\!\!\!\perp T \mid R \rightarrow$ 独立 = 非激活

$B \perp\!\!\!\perp T \mid R \rightarrow$ 不独立 = 激活

$\rightarrow$ 非激活路径

$\rightarrow L \perp\!\!\!\perp B \mid R$ 成立!



构造贝叶斯网络的方法

构造一个贝叶斯网络，并使其产生的联合分布能很好地表示给定的域。

1. 确定建模域所需的变量集合并排序，得到 $X_1, \dots, X_n$
2. i 从 1 到 n ，循环执行
 - 将 X_i 添加到网络
 - 从 $X_1, \dots, X_{i-1}$ 选择父节点使得
$$P(X_i | Parents(X_i)) = P(X_i | X_1, \dots, X_{i-1})$$

构造贝叶斯网络的方法

构造一个贝叶斯网络，并使其产生的联合分布能很好地表示给定的域。

1. 确定建模域所需的变量集合并排序，得到 $X_1, \dots, X_n$
2. i 从 1 到 n ，循环执行
 - 将 X_i 添加到网络
 - 从 $X_1, \dots, X_{i-1}$ 选择父节点使得
$$P(X_i | Parents(X_i)) = P(X_i | X_1, \dots, X_{i-1})$$

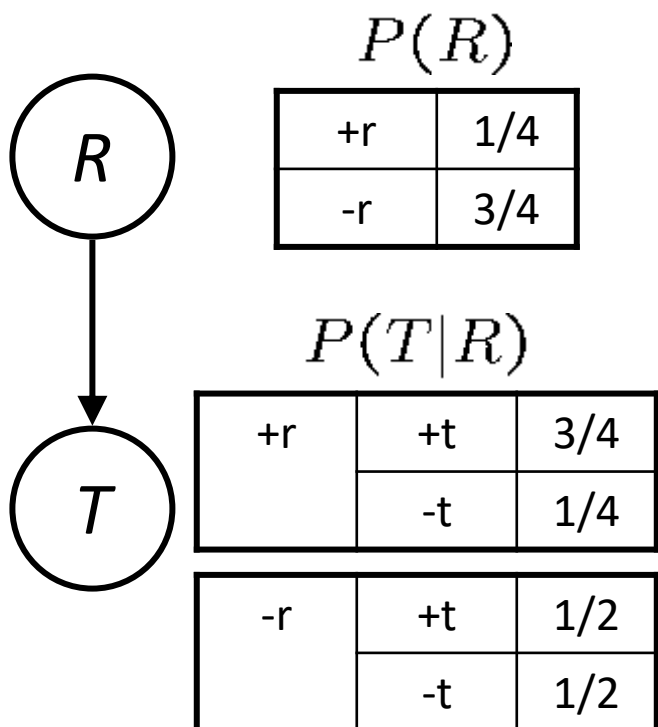
通过因果关系来定义有向边。

父节点的选择应该保证全局语义：

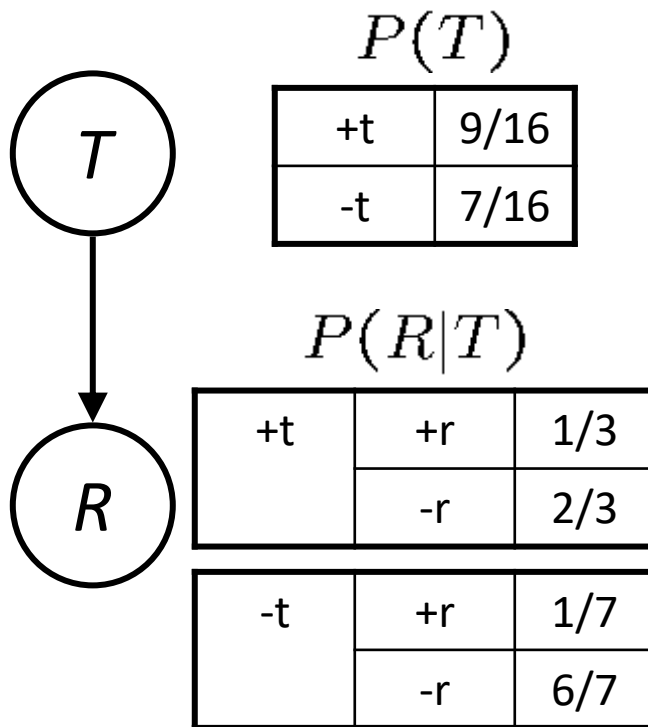
$$\begin{aligned} P(X_1, \dots, X_n) &= \prod_{i=1}^n P(X_i | X_1, \dots, X_{i-1}) && \text{(链式法则)} \\ &= \prod_{i=1}^n P(X_i | Parents(X_i)) && \text{(根据构建过程)} \end{aligned}$$

贝叶斯网络中的因果关系

因果关系



反向因果关系



$P(T, R)$

+r	+t	3/16
+r	-t	1/16
-r	+t	6/16
-r	-t	6/16

贝叶斯网络中的因果关系

- 当贝叶斯网络反映真实的因果关系时:
 - 通常更简单 (节点有着更少的父节点)
 - 通常更容易理解
 - 通常更容易从领域专家获得建议

贝叶斯网络中的因果关系

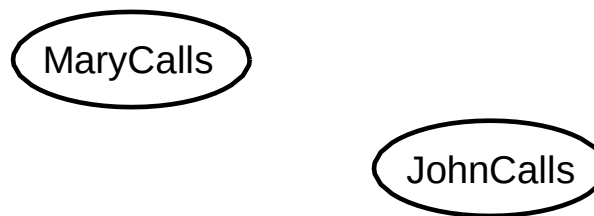
- 当贝叶斯网络反映真实的因果关系时:
 - 通常更简单 (节点有着更少的父节点)
 - 通常更容易理解
 - 通常更容易从领域专家获得建议
- 贝叶斯网络并不一定需要因果关系
 - 在领域中因果关系不一定存在(特别是当有变量缺失的时候)
 - 箭头只反映相关性, 不反映因果
- 箭头真正的含义?
 - 拓扑图可能表示因果关系
 - 拓扑图真正表达的是条件独立性

$$P(x_i|x_1, \dots, x_{i-1}) = P(x_i|\text{parents}(X_i))$$

构造贝叶斯网络的方法

范例

假设按照 M, J, A, B, E 的顺序添加节点

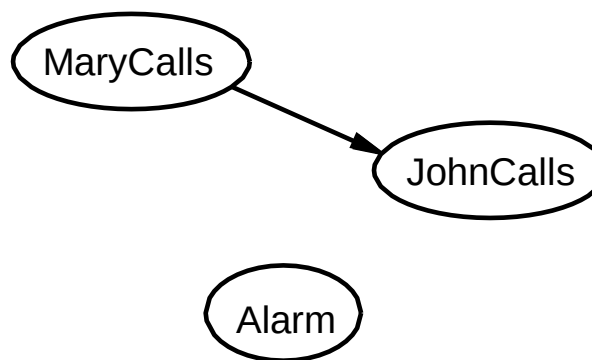


$$P(J|M) = P(J)?$$

构造贝叶斯网络的方法

范例

假设按照 M, J, A, B, E 的顺序添加节点



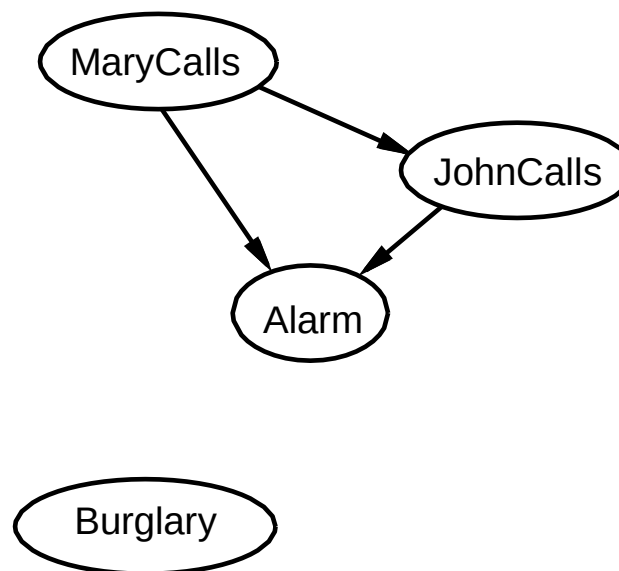
$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$?

构造贝叶斯网络的方法

范例

假设按照 M, J, A, B, E 的顺序添加节点



$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

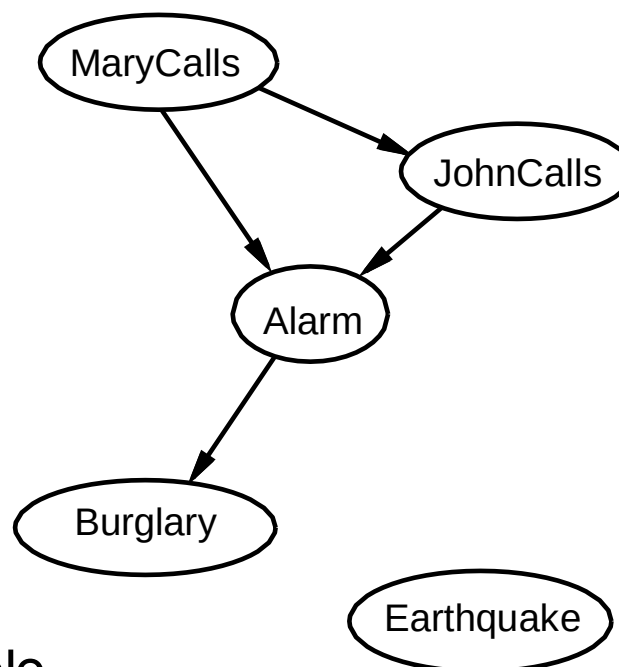
$P(B|A, J, M) = P(B|A)$?

$P(B|A, J, M) = P(B)$?

构造贝叶斯网络的方法

范例

假设按照 M, J, A, B, E 的顺序添加节点



$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

$P(B|A, J, M) = P(B|A)$? Yes

$P(B|A, J, M) = P(B)$? No

$P(E|B, A, J, M) = P(E|A)$?

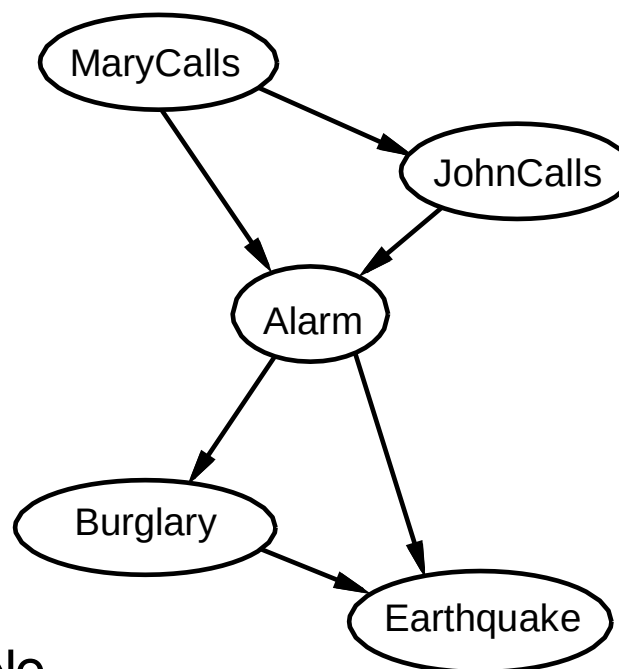
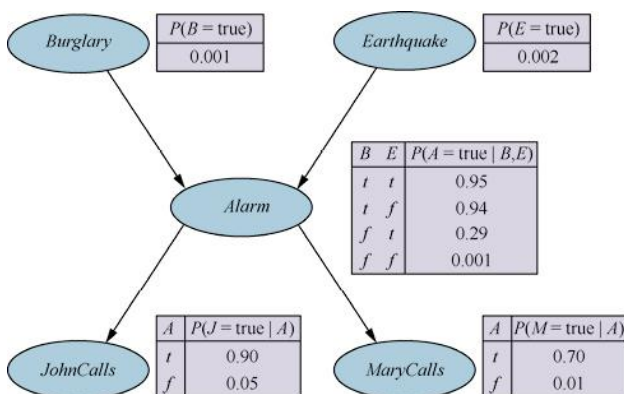
$P(E|B, A, J, M) = P(E|A, B)$?

构造贝叶斯网络的方法

范例

假设按照 M, J, A, B, E 的顺序添加节点

原始网络



$P(J|M) = P(J)$? No

$P(A|J, M) = P(A|J)$? $P(A|J, M) = P(A)$? No

$P(B|A, J, M) = P(B|A)$? Yes

$P(B|A, J, M) = P(B)$? No

$P(E|B, A, J, M) = P(E|A)$? No

$P(E|B, A, J, M) = P(E|A, B)$? Yes

任何**概率推理**系统的基本任务都是给定一些观测到的**事件 (event)**，通常是一组**证据变量 (evidence variable)** 的赋值，计算一组**查询变量 (query variable)** 的后验概率分布。

简单查询: 计算后验边缘概率 $P(X_i | E = e)$

例如, $P(\text{NoGas} | \text{Gauge} = \text{empty}, \text{Lights} = \text{on}, \text{Starts} = \text{false})$

联合查询: $P(X_i, X_j | E = e) = P(X_i | E = e)P(X_j | X_i, E = e)$

最优决策: 决策网络包括效用信息；所需的概率推理 $P(\text{outcome} | \text{action}, \text{evidence})$ ，从而计算期望效用值。

解释: $\text{argmax}_q P(Q = q | E_1 = e_1 \dots)$

贝叶斯网络中的精确推断

通过枚举进行推断

- 一般形式:
 - 证据变量: $E_1 \dots E_k = e_1 \dots e_k$
 - 查询变量: Q
 - 隐藏变量: $H_1 \dots H_r$ $\left. \begin{array}{l} E_1 \dots E_k = e_1 \dots e_k \\ Q \\ H_1 \dots H_r \end{array} \right\} \begin{array}{l} X_1, X_2, \dots, X_n \\ \text{所有变量} \end{array}$
- 目标概率: $P(Q|e_1 \dots e_k)$

贝叶斯网络中的精确推断

通过枚举进行推断

- 一般形式:

- 证据变量: $E_1 \dots E_k = e_1 \dots e_k$
 - 查询变量: Q
 - 隐藏变量: $H_1 \dots H_r$
- $$\left. \begin{array}{l} \\ \\ \end{array} \right\} \begin{array}{l} X_1, X_2, \dots, X_n \\ \text{所有变量} \end{array}$$

- 目标概率: $P(Q|e_1 \dots e_k)$

- **步骤1:** 在联合概率分布表中选择和证据变量一致的条目。

- **步骤2:** 对隐藏变量H求和消元来获得查询和证据变量的联合概率。

$$P(Q, e_1 \dots e_k) = \sum_{h_1 \dots h_r} \underbrace{P(Q, h_1 \dots h_r, e_1 \dots e_k)}_{X_1, X_2, \dots, X_n}$$

- **步骤3:** 归一化。

$$P(Q|e_1 \dots e_k) = \frac{1}{Z} P(Q, e_1 \dots e_k) \quad Z = \sum_q P(Q, e_1 \dots e_k)$$

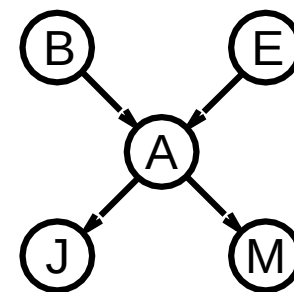
贝叶斯网络中的精确推断

通过枚举进行推断

通过计算贝叶斯网络中条件概率的乘积之和回答查询。

关于 burglary 网络的简单查询：

- 证据变量: J, M
- 查询变量: B
- 隐藏变量: A, E



$$\begin{aligned} P(B|j, m) \\ &= P(B, j, m) / P(j, m) \\ &= a P(B, j, m) \\ &= a \sum_e \sum_a P(B, e, a, j, m) \end{aligned}$$

使用条件分布重写完全联合分布：

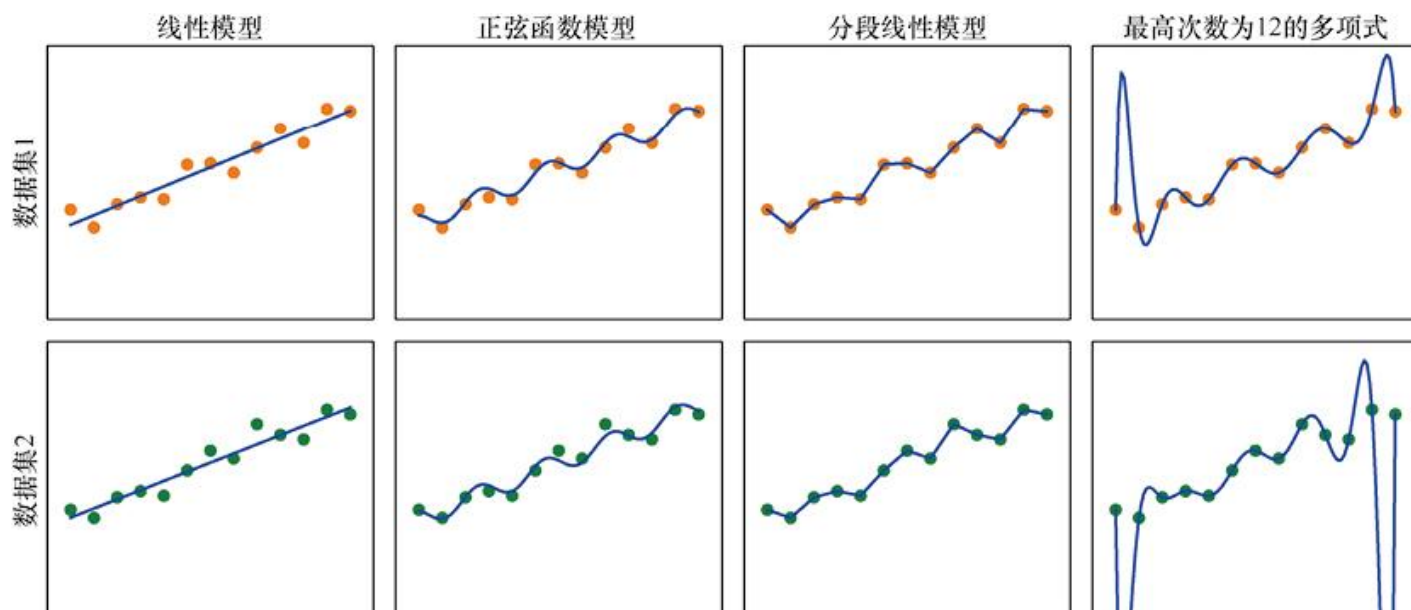
$$\begin{aligned} P(B|j, m) \\ &= a \sum_e \sum_a P(B) P(e) P(a|B, e) P(j|a) P(m|a) \\ &= a P(B) \sum_e P(e) \sum_a P(a|B, e) P(j|a) P(m|a) \end{aligned}$$

◆ 第十九章：样例学习

- 监督学习
- 模型选择与模型优化
- 线性回归与分类
- 非参数模型：最近邻模型

- 伴随输入有3种类型的反馈（**feedback**），它们决定了3种类型的学习：
 - 监督学习
 - 智能体观测到输入-输出对
 - 学习从输入到输出的一个函数映射
 - 无监督学习
 - 智能体从没有任何显式反馈的输入中学习模式
 - 聚类（**clustering**）：通过输入样例来检测潜在的有价值的聚类簇
 - 强化学习
 - 智能体从一系列强化（奖励和惩罚）中学习

- 给定一个训练集含有 N 个“输入-输出”对样例：
 - $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N),$
 - $y = f(x)$
- 函数 h 被称为关于世界的假设来近似真实的函数 f
 - 取自一个假设空间 H ，其中包含所有可能的函数
 - h 是关于数据的模型,取自模型类 H
- 一致性假设: 假设 h 使得训练集中任意一个 x_i 都有 $h(x_i) = y_i$.
 - 寻找最佳拟合函数使得 $h(x_i)$ 接近于 y_i
- 衡量一个假设的标准不是看它在训练集上的表现，而是取决于它如何处理尚未观测到的输入。
 - 使用一个测试集 (x_i, y_i) 来评估假设，如果 h 准确地预测了测试集的输出，我们称 h 具有很好的泛化能力。



寻找拟合数据的假设。第一行：在数据集1上训练的来自4个不同假设空间的最佳拟合函数的4个图像。第二行：同样的4个函数，但是在稍有不同的数据集上进行训练得到的结果（数据集采样自相同的函数 $f(x)$ ）

- 使用**偏差 (bias)** 分析假设空间
 - 在不同的训练集上，假设所预测的值偏离期望值的平均趋势。
- **欠拟合 (underfitting)**：无法找到数据中的模式。
- **方差 (variance)**：指由训练数据波动而导致假设的变化量。
- **过拟合 (overfitting)**：当一个函数过于关注它用来训练的特定训练数据集，进而导致它在没有见过的数据上表现较差。
- **偏差-方差权衡 (bias-variance tradeoff)**：在更复杂、低偏差的能较好拟合训练集的假设与更简单、低方差的可能泛化得更好的假设中做出选择。
 - 选择与数据相符的最简单的假设。
 - **奥卡姆剃刀原则 (Ockham's razor)**：如无必要，勿增实体。

- 相比于仅仅判断一个假设是可能还是不可能的，给出一个假设可能发生的概率是更合理的策略。
- 给定数据集，概率最大的假设 h^* ：

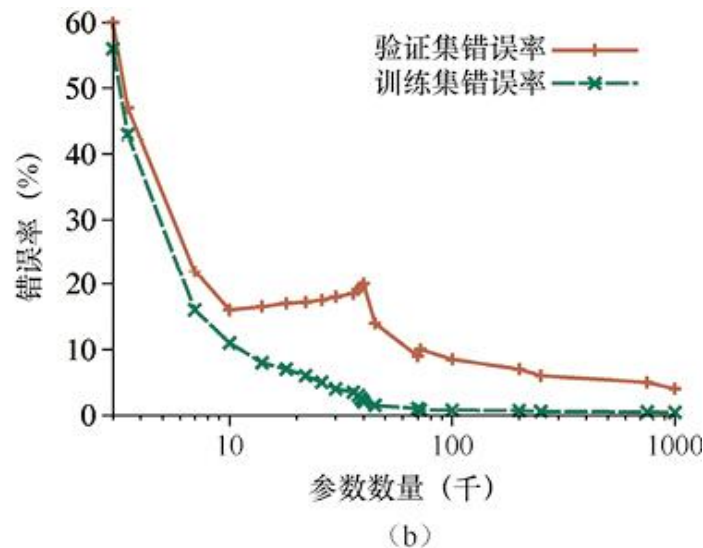
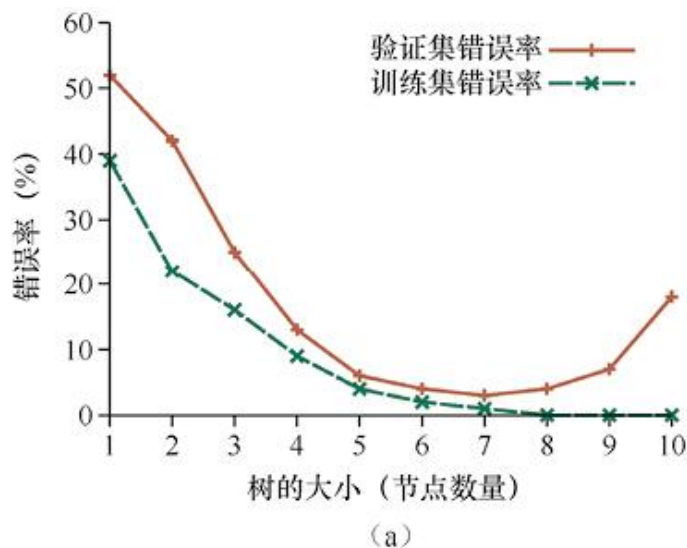
$$h^* = \operatorname{argmax}_{h \in H} P(h|data)$$

- 根据贝叶斯法则，上式等价于

$$h^* = \operatorname{argmax} P(data|h) P(h)$$

- 找到一个好的假设这一目标可以分作两个子任务：
 - **模型选择**：选择一个好的假设空间；
 - **模型优化 (训练)**：在这个空间中找到最佳假设。
- 一组为用于训练从而得到假设的训练集，另一组为用于评估假设的测试集。
- **错误率**：对于样本 (x, y) ， $h(x) \neq y$ 的比例
- 需要三个数据集：
 1. 训练集用于训练备选模型
 2. 验证集也被称为开发集，用于评估备选模型并选择最佳的备选模型
 3. 测试集用于无偏地估计最佳模型
- 当没有足够的数据来构成这3个数据集：**k-折交叉验证**
 - 将数据分割成k个相等大小的子集；
 - 然后进行k轮学习；
 - 在每一轮中， $1/k$ 的数据被作为验证集，其余的样例被用于训练；
 - 常用k值为5或10
 - **留一交叉验证**： $k=n$

模型选择



两个不同问题上不同复杂性模型的训练误差（下方绿线）和验证误差（上方橙色线）

线性回归与分类

单变量线性回归

输入 x 和输出 y

$$y = w_1x + w_0$$

线性函数

$$h_w(x) = w_1x + w_0$$

线性回归：找到最匹配这些数据的线性函数 h_w

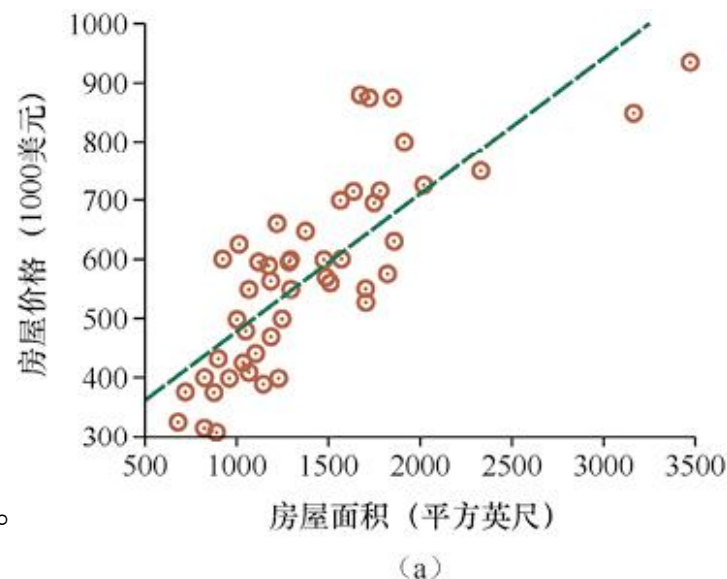
找到对应的权重值 (w_0, w_1) 使得其经验损失最小。

平方误差损失函数, L_2 , 对所有训练样本求和：

$$Loss(h_w) = \sum_{j=1}^N L_2(y_j, h_w(x_j)) = \sum_{j=1}^N (y_j - h_w(x_j))^2 = \sum_{j=1}^N (y_j - (w_1x_j + w_0))^2.$$

$$\frac{\partial}{\partial w_0} \sum_{j=1}^N (y_j - (w_1x_j + w_0))^2 = 0 \text{ and } \frac{\partial}{\partial w_1} \sum_{j=1}^N (y_j - (w_1x_j + w_0))^2 = 0.$$

$$w_1 = \frac{N(\sum x_j y_j) - (\sum x_j)(\sum y_j)}{N(\sum x_j^2) - (\sum x_j)^2}; \quad w_0 = (\sum y_j - w_1(\sum x_j))/N.$$



梯度下降

- 通过逐步修改参数来搜索连续的权重空间（最小化损失函数）

$\mathbf{w} \leftarrow$ any point in the parameter space

while not converged do

for each w_i in $\mathbf{w}$ do

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(\mathbf{w})$$

- α : 步长/学习率，可以是固定的常数，也可以随着学习过程的进行逐渐衰减。

$$\begin{aligned} \frac{\partial}{\partial w_i} \text{Loss}(\mathbf{w}) &= \frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(x))^2 = 2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(x)) \\ &= 2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i} (y - (w_1 x + w_0)). \end{aligned}$$

$$\frac{\partial}{\partial w_0} \text{Loss}(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)) \quad \frac{\partial}{\partial w_1} \text{Loss}(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)) \times x.$$

线性回归与分类

梯度下降

批量梯度下降（Batch Gradient Descent）

- 最小化每个样例各自损失的总和

$$w_0 \leftarrow w_0 + \alpha \sum_j (y_j - h_{\mathbf{w}}(x_j)); \quad w_1 \leftarrow w_1 + \alpha \sum_j (y_j - h_{\mathbf{w}}(x_j)) \times x_j.$$

- 损失曲面是凸的
- 训练不会陷入局部极小值，到全局最小值的收敛性可以保证
- 轮（epoch）：遍历了所有训练样例的一步更新

随机梯度下降（Stochastic gradient descent or SGD）

- 在每一步中随机选择少量训练样例
- 是从N个样例中选择一个大小为m的小批量（minibatch）样例
- 选择合适的m来利用并行向量运算

参数模型：通过参数个数固定的参数集（与训练样例的数量无关）来汇总数据信息的学习模型。

非参数模型：无法用参数个数固定的参数集来表示的模型

例如，分段线性函数将所有数据点保留为模型的一部分，也称为基于实例的学习（instance-based learning）或基于记忆的学习（memory-based learning）。

最简单的基于实例的学习方法是**查表**

- 取所有的训练样例，将它们放在查找表中，然后在需要访问 $h(\mathbf{x})$ 时，查看 $\mathbf{x}$ 是否在表中；如果是，则返回相应的 y .

最近邻模型

k -近邻

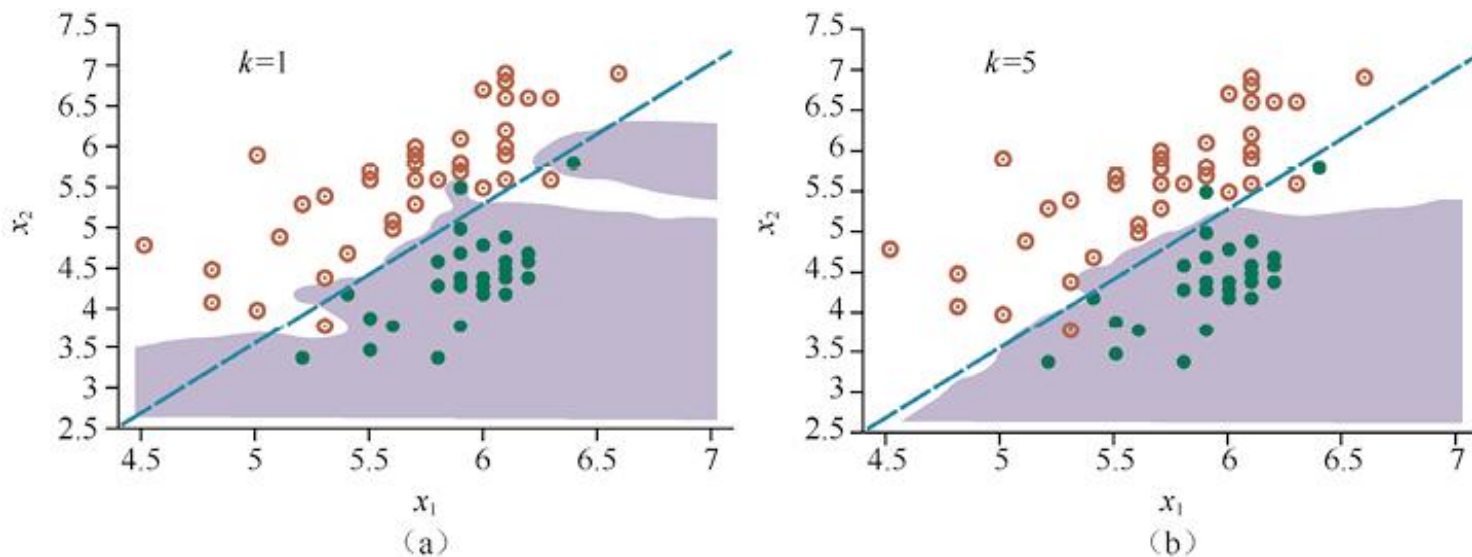
- 给定待查询的 $\mathbf{x}_q$, 寻找最接近 $\mathbf{x}_q$ 的 k 个样例。
- 为了实现分类, 我们寻找 $\mathbf{x}_q$ 的一组邻居, 并以占比最大的输出值为分类结果。例如 $k = 3$ 并且输出值为 *Yes, No, Yes*, 则分类结果为 *Yes*。
- 为了实现回归, 我们可以取 k 个邻居的平均值或中位数, 也可以在最近邻邻居上求解一个线性回归问题。
- 闵可夫斯基距离 (Minkowski distance) 或 L^p 范数

$$L^p(\mathbf{x}_j, \mathbf{x}_q) = \left(\sum_i |x_{j,i} - x_{q,i}|^p \right)^{1/p}.$$

- $p=2$, 欧几里得距离
- $p=1$, 曼哈顿距离
- 对于布尔属性值, 汉明距离
- 马氏距离: 考虑维度之间的协方差

$$D_M(x, y) = \sqrt{(x - y)^T \Sigma^{-1} (x - y)}$$

最近邻模型



(a) 一个 k 近邻模型，该图显示了地震数据中的爆炸一类的范围，其中 $k=1$ 。可以明显看到有过拟合现象。(b) 当 $k=5$ 时，该数据集上的过拟合现象消失了

知识点回顾

◆ 第二十章：概率模型学习

- 贝叶斯学习
- 最大后验学习
- 最大似然学习
- 最大似然参数学习：离散模型

贝叶斯学习

- 假设有五种糖果袋:

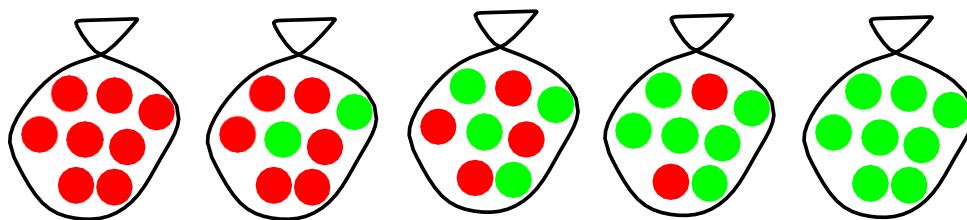
10% 为 h_1 : 100% 樱桃味

20% 为 h_2 : 75% 樱桃味 + 25% 酸橙味

40% 为 h_3 : 50% 樱桃味 + 50% 酸橙味

20% 为 h_4 : 25% 樱桃味 + 75% 酸橙味

10% 为 h_5 : 100% 酸橙味



- 然后我们观测从某个袋子中抽取的糖果: ● ● ● ● ● ● ● ● ● ●

- 这是什么种类的袋子? 下一块糖果是什么口味?

贝叶斯学习

- 将学习看作假设空间中概率分布的贝叶斯更新:

H 是假设变量, 值为 $h_1, h_2, \dots$, 先验分布 $P(H)$

第 j 个观测 d_j 给出了随机变量的输出 D_j

训练数据 $d = d_1, \dots, d_N$

- 给定到目前为止的数据, 每一个假设有一个后验分布:

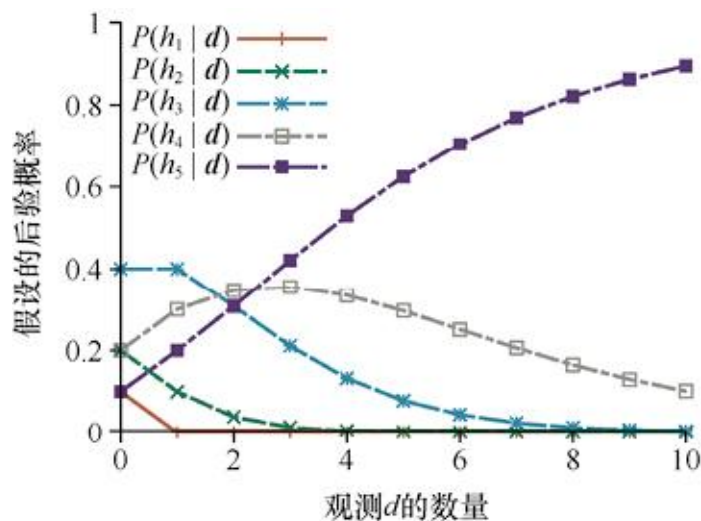
$$P(h_i|d) = \frac{1}{a} P(d|h_i)P(h_i)$$

这里 $P(d|h_i)$ 被称为似然

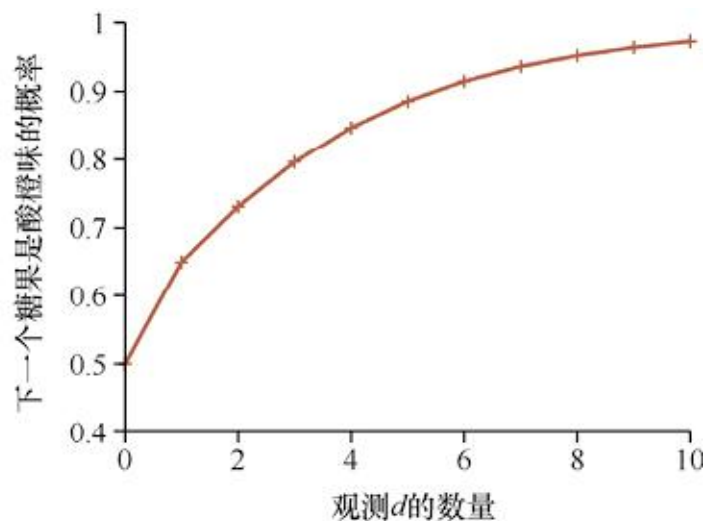
- 预测为在假设上的概率加权平均:

$$P(X|d) = \sum_i P(X|d, h_i)P(h_i|d) = \sum_i P(X|h_i)P(h_i|d)$$

贝叶斯学习



(a)



(b)

图20-1 (a) 根据式 (20-1) 得到的后验概率 $P(h_i | d_1, \dots, d_N)$ 。观测数量 N 为1~10, 且每一个观测都是酸橙味的糖果。 (b) 基于式 (20-2) 的贝叶斯预测 $P(D_{N+1} = \text{lime} | d_1, \dots, d_N)$

$$P(\mathbf{d} | h_i) = \prod_j P(d_j | h_i) \quad P(h_i | \mathbf{d}) = \alpha P(\mathbf{d} | h_i) P(h_i)$$

最大后验学习

- 在假设空间上求和通常是非常困难的 $\sum_i P(X|h_i)P(h_i|d)$
- 最大后验 (MAP)学习: 选择 h_{MAP} 来最大化 $P(h_i|d)$

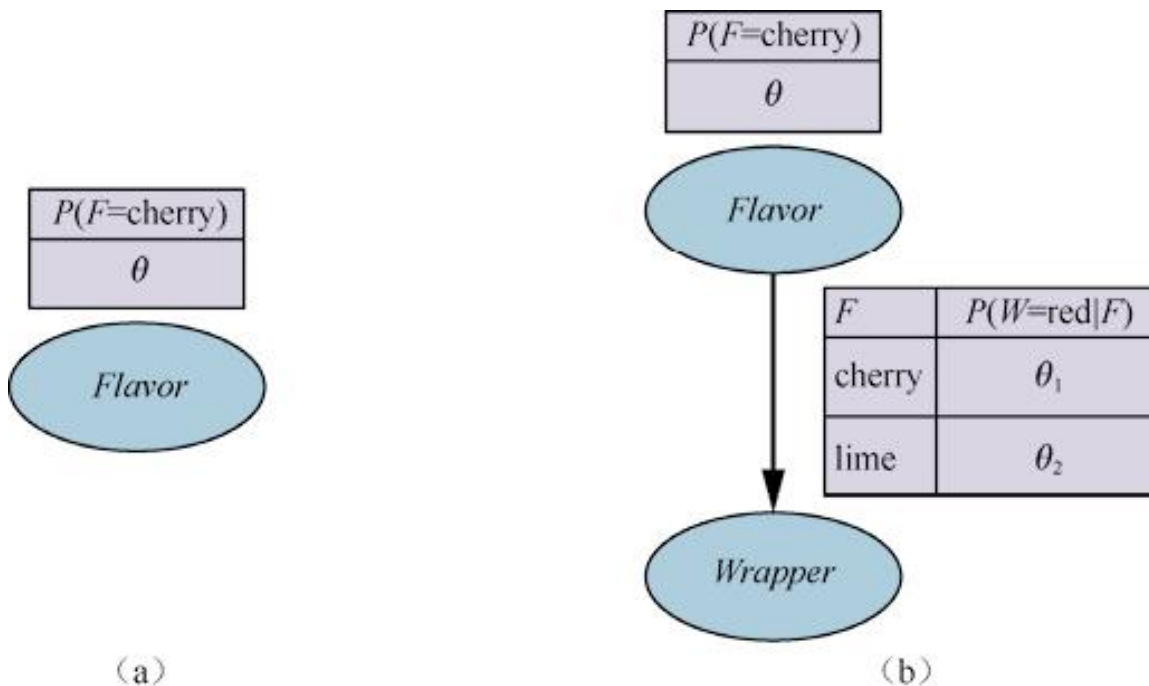
即, 最大化 $P(d|h_i)P(h_i)$ 或 $\log P(d|h_i) + \log P(h_i)$

$$P(X|d) \approx P(X|h_{\text{MAP}})$$

最大似然学习

- 当数据集很大时，假设的先验分布就不那么重要了，因为来自数据的证据足够强大，足以淹没假设的先验分布。
- 最大似然 (ML) 学习: 选择 h_{ML} 来最大化 $P(d|h_i)$
- 即, 简单的获取对数据的最佳拟合; 对于假设空间具有均匀先验分布, 等同于最大后验学习 (例如所有的假设都同样复杂)
- 最大似然学习是“标准”的（非贝叶斯）统计学习方法

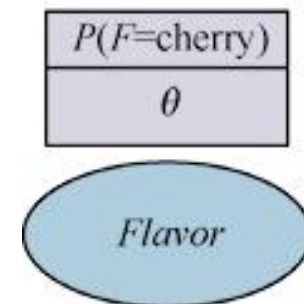
最大似然参数学习：离散模型



(a) 樱桃味糖果和酸橙味糖果比例未知情况下的贝叶斯网络。(b) 包装颜色（依概率）与糖果口味相关情况下的模型

最大似然参数学习：离散模型

- 可能含有樱桃味和酸橙味糖果的糖果袋，其中糖果口味的比例完全未知。
- 参数 θ 表示樱桃味糖果所占的比例，其对应的假设为 h_θ



最大似然参数学习：离散模型

- 可能含有樱桃味和酸橙味糖果的糖果袋，其中糖果口味的比例完全未知。
- 参数 θ 表示樱桃味糖果所占的比例，其对应的假设为 h_θ
- 如果我们假设所有的比例有相同的先验可能性，那么采用**最大似然估计**是合理的
- 现在假设我们已经打开了 N 颗糖果，其中有 c 颗为樱桃味，则该特定数据集的似然为：

$$P(\mathbf{d} | h_\theta) = \prod_{j=1}^N P(d_j | h_\theta) = \theta^c \cdot (1-\theta)^\ell$$

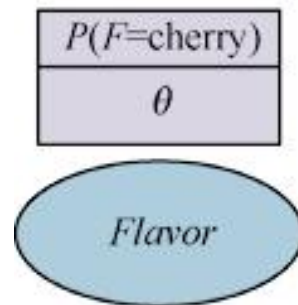
- 最大似然假设所需的参数即为使得上式最大化的参数。由于 $\log$ 函数是单调函数，我们可以最大化对数似然来简化计算：

$$L(\mathbf{d} | h_\theta) = \log P(\mathbf{d} | h_\theta) = \sum_{j=1}^N \log P(d_j | h_\theta) = c \log \theta + \ell \log (1-\theta)$$

- 对上式关于 θ 进行求导，并令导数为0可得：

$$\frac{dL(\mathbf{d} | h_\theta)}{d\theta} = \frac{c}{\theta} - \frac{\ell}{1-\theta} = 0 \quad \Rightarrow \quad \theta = \frac{c}{c+\ell} = \frac{c}{N}$$

- 验证二阶导数小于0，因此该极值确实是最大值。



最大似然参数学习：离散模型

- 最大似然参数学习的标准流程：
 - (1) 将数据的似然写成关于参数的函数的形式。
 - (2) 计算对数似然关于每个参数的导数。
 - (3) 解出使得导数为0的参数。
 - (4) 验证二阶导数是否小于0。
- 最大似然学习中普遍存在的一个重要问题：当数据集非常小以至于一些事件还未发生时，最大似然假设将把这些事件的概率置为0。

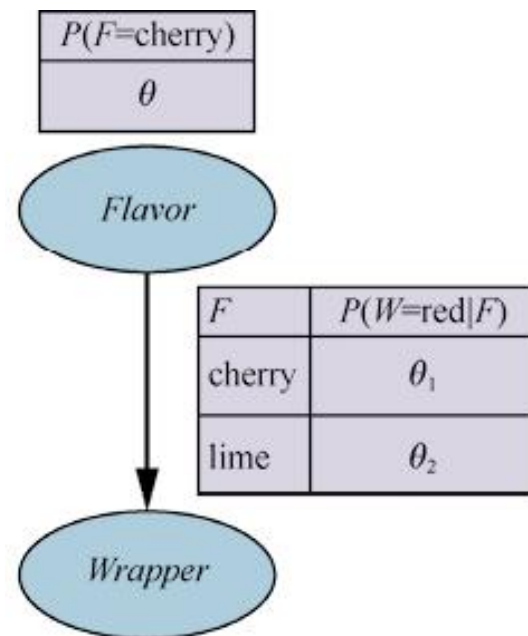
最大似然参数学习：离散模型

- 红、绿两种不同颜色的糖果包装；其包装在概率上服从某个条件分布，该分布取决于糖果的口味。
- 观测到一颗带有绿色包装的樱桃味糖果的似然：

$$\begin{aligned} P(\text{Flavor} = \text{cherry}, \text{Wrapper} = \text{green} \mid h_{\theta, \theta_1, \theta_2}) \\ &= P(\text{Flavor} = \text{cherry} \mid h_{\theta, \theta_1, \theta_2}) P(\text{Wrapper} = \text{green} \mid \text{Flavor} = \text{cherry}, h_{\theta, \theta_1, \theta_2}) \\ &= \theta \cdot (1 - \theta_1) \end{aligned}$$

- N 颗糖果， r_c 颗包装为红色的樱桃味糖果等等，则该数据的似然为：

$$P(\mathbf{d} \mid h_{\theta, \theta_1, \theta_2}) = \theta^c (1 - \theta)^{\ell} \cdot \theta_1^{r_c} (1 - \theta_1)^{g_c} \cdot \theta_2^{r_l} (1 - \theta_2)^{g_l}$$



最大似然参数学习：离散模型

- 对似然取对数：

$$L = [c \log \theta + \ell \log(1 - \theta)] + [r_c \log \theta_1 + g_c \log(1 - \theta_1)] + [r_\ell \log \theta_2 + g_\ell \log(1 - \theta_2)]$$

- 对数似然的具体形式是3项求和，其中每一项包含单独的一个参数。
- 令对数似然对每个参数求导并置为0时，可以得到3个独立的方程，其中每一个方程只含有一个参数：

$$\frac{\partial L}{\partial \theta} = \frac{c}{\theta} - \frac{\ell}{1 - \theta} = 0 \quad \Rightarrow \quad \theta = \frac{c}{c + \ell}$$

$$\frac{\partial L}{\partial \theta_1} = \frac{r_c}{\theta_1} - \frac{g_c}{1 - \theta_1} = 0 \quad \Rightarrow \quad \theta_1 = \frac{r_c}{r_c + g_c}$$

$$\frac{\partial L}{\partial \theta_2} = \frac{r_\ell}{\theta_2} - \frac{g_\ell}{1 - \theta_2} = 0 \quad \Rightarrow \quad \theta_2 = \frac{r_\ell}{r_\ell + g_\ell}$$

- 一旦有了完全数据，贝叶斯网络的最大似然参数学习问题将可以被分解为一些分离的学习问题，每个问题对应一个参数。

